

Q&A Based Interactive Storytelling Using Generative AI

A

Dissertation Report

Submitted for Partial fulfillment of the Requirement

For the award of the degree of

MASTER OF TECHNOLOGY

In

COMPUTER ENGINEERING

Submitted by

Anjali Madan Jha

(Roll No. 124MTCM1008)

Under the guidance of

Dr. Vidyullata Devmane

Professor



DEPARTMENT OF COMPUTER ENGINEERING

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)

MUMBAI - 400 088, MAHARASHTRA (INDIA)

2025-2026



SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)

MUMBAI - 400 088, MAHARASHTRA (INDIA)

CERTIFICATE

It is my pleasure to certify that **Anjali Madan Jha** worked under my supervision for the M. Tech Dissertation-II report entitled **Q&A Based Interactive Storytelling Using Generative AI** and her work is of the level of requirement set up for the Dissertation report in **Computer Engineering** by Shah & Anchor Kutchhi Engineering College, Mumbai.

Date:

Anjali Madan Jha

Roll No: 124MTCM1008

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dr. Vidyullata Devmane

Professor

Department of Computer Engineering

Shah & Anchor Kutchhi Engineering College

Mumbai – 400 088, India

The viva-voce/oral and internal practical examination of **Anjali Madan Jha**, M. Tech. in Computer Engineering, has been held on

External Examiner

Internal Examiner

Guide Name

Head of Department

Principal

College Seal

CANDIDATE'S DECLARATION

I hereby declare that the work presented in the Dissertation report entitled “**Q&A Based Interactive Storytelling Using Generative AI**” for partial fulfillment of the requirement for the degree of M. Tech. in Computer Engineering and submitted to the Department of Computer Engineering at Shah & Anchor Kutchhi Engineering College, Mumbai, is an authentic record of my own work carried out during the period from August 2025 to May 2026 under the supervision of **Dr. Vidyullata Devmane**

The matter presented in this Dissertation report has not been submitted elsewhere in part or fully to any other University or Institute for the award of any other degree.

Signature of the student

Name: Anjali Madan Jha

Roll No: 124MTCM1008



SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)

CERTIFICATE OF PLAGIARISM CHECK	
Name of the Student	Anjali Madan Jha
Title of the Dissertation/Report	Q&A Based Interactive Storytelling Using Generative AI
Name of the Supervisor/Guide	Dr. Vidyullata Devmane
Name of the Department	Department of Computer Engineering
Similar content (%) identified	%
Acceptable Maximum Limit	10%
Name of the Similarity tool used for Plagiarism Report	Turnitin
Date of Verification	
Name & Sign of the Department Academic Integrity Panel (DAIP) Member	
Name & Sign of the Student	Anjali Madan Jha
Name & Sign of the Guide	Dr. Vidyullata Devmane

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my guide, **Dr. Vidyullata Devmane**, Professor, Department of Computer Engineering, for her guidance, patience and steady encouragement throughout this dissertation. Her suggestions at every stage, from framing the problem to reviewing the final results, shaped both the research questions and the design of the system.

I thank the Head of the Department of Computer Engineering, the Project Review Committee and all the faculty members for their suggestions during the periodic reviews. Their questions on the evaluation method and on the multimodal pipeline improved this work considerably. I am grateful to the Principal and the management of Shah & Anchor Kutchhi Engineering College for providing a good environment for research.

I also thank the fifteen volunteers who took part in the user study, and the developers of the open-source tools on which this project is built, including FastAPI, SQLite, Ollama, Groq, Pollinations.ai and D3.js.

Finally, I thank my family and friends for their constant support and understanding, without which this work would not have been possible.

Anjali Madan Jha

Shah & Anchor Kutchhi Engineering College

Date:

ABSTRACT

Large Language Models (LLMs) such as GPT-4, LLaMA and Mistral can now write fluent and creative stories, but most story tools still work in a single step: the user types one prompt and receives one finished story. The user has to state every preference at the start, and has little say in how the story develops. This makes the output generic and leaves the reader as a passive consumer.

This dissertation presents a question-and-answer (Q&A) based interactive storytelling system in which the user and the model build the story together. A structured protocol of 17 questions, grouped into four narrative dimensions (setting, character, theme and plot), collects the user's preferences. The user can choose a budget of 4, 8, 12 or 17 questions, write a custom answer to any question, and add new questions of their own. The answers pass through a seven-module backend: User Input Acquisition, Context Extraction, Knowledge and Memory, Prompt Engineering, LLM-based Story Generation, Story Flow and Consistency Management, and Output Formatting. A five-phase story-flow model based on Freytag's pyramid keeps the plot moving from introduction to resolution. The backend is built with FastAPI and SQLite and works with local (Ollama) and cloud (Groq, Hugging Face, OpenAI) language models, which can be switched at run time.

The text is supported by three further modes. A scene illustration is generated for each segment through the Pollinations.ai diffusion service, using a sentence-scoring prompt extractor and a serial request queue that avoids rate-limit failures. Narration is produced with the W3C Web Speech Synthesis interface, and genre-matched ambient music is streamed with cross-fades. Readers can explore the story through a D3-based Story Map and a Character Relationship Graph built from co-appearance counts.

The system was evaluated on 50 generated stories and a user study with 15 participants. It obtained mean Likert scores of 4.05/5 for coherence, 4.12/5 for creativity, 4.35/5 for engagement and 3.75/5 for consistency, with an overall satisfaction of 4.3/5. Groq LLaMA-3.1-8B produced the first story segment in 1.8 s on average. The serial queue cut image-generation failures under parallel load from about 70% to under 2%, and filtering raised the character-detection F_1 score from 0.71 to 0.80. A budget of 12 questions gave the best balance between personalisation and the time spent answering.

Keywords: Generative AI, Large Language Models, interactive storytelling, question answering, narrative generation, prompt engineering, text-to-image synthesis, text-to-speech, human-AI collaboration, FastAPI

Table of Contents

Certificate	ii
Candidate’s Declaration	iii
Certificate of Plagiarism Check	iv
Acknowledgements	v
Abstract	vi
List of Figures	x
List of Tables	xi
1. Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Objectives of the Study	3
1.5 Research Questions	5
1.6 Scope of the Work	5
1.7 Significance of the Study	5
1.8 Contributions	6
1.9 Structure of the Report	7
2. Literature Review	8
2.1 Introduction	8
2.2 Overview of the Topic	8
2.2.1 Evolution of Automated Story Generation	8
2.2.2 Interactive Storytelling	9
2.2.3 Multimodal Storytelling	9
2.3 Theoretical Framework	10
2.3.1 Narrative Structure	10
2.3.2 Transformer Language Models	10
2.3.3 Prompt Engineering	11
2.3.4 Text-to-Image Diffusion	11
2.3.5 Speech Synthesis and Ambient Music	11
2.4 Previous Research	12
2.4.1 Controllable and Planned Story Generation	12
2.4.2 Interactive and Co-Creative Systems	12
2.4.3 Multimodal Story Generation	13
2.4.4 Evaluation of Generated Stories	13
2.4.5 Summary of Reviewed Literature	13

2.5	Research Gaps	14
2.6	Summary	16
3.	Proposed Methodology and System Design	17
3.1	Introduction	17
3.2	Feasibility Study	17
3.2.1	Technical Feasibility	17
3.2.2	Economic Feasibility	18
3.2.3	Operational Feasibility	18
3.2.4	Legal and Ethical Feasibility	18
3.3	Project Planning and Scheduling	18
3.4	System Overview	20
3.5	Research Design	21
3.6	Question-and-Answer Framework	22
3.6.1	Question Pool	22
3.6.2	Elicitation Protocol	23
3.6.3	Context Projection and Prompt Construction	23
3.7	Five-Phase Story Progression	24
3.8	Generation Loop	25
3.9	Backend Modules	26
3.9.1	Module 1: User Input Acquisition	26
3.9.2	Module 2: Context Extraction Engine	28
3.9.3	Module 3: Knowledge and Memory	28
3.9.4	Module 4: Prompt Engineering	29
3.9.5	Module 5: Story Generator	29
3.9.6	Module 6: Story Flow Manager	30
3.9.7	Module 7: Output Formatter	30
3.10	Multimodal Pipelines	31
3.10.1	Scene Illustration	31
3.10.2	Narration and Ambient Music	33
3.11	Reader Analytics	34
3.11.1	Story Map	34
3.11.2	Character Relationship Graph	34
3.12	System Implementation	34
3.12.1	Project Structure	34
3.12.2	Database Schema	35
3.12.3	REST API	35
3.12.4	Technology Stack	36
3.12.5	Hardware Requirements	37
3.12.6	Implementation Size	37

3.13	Evaluation Method	38
3.14	Ethical Considerations	39
3.15	Summary	39
4.	Results and Discussions	40
4.1	Results	40
4.1.1	Introduction	40
4.1.2	Experimental Setup	40
4.1.3	Software Testing	41
4.1.3.1	Unit Testing	41
4.1.3.2	Integration and API Testing	41
4.1.3.3	User Interface and Robustness Testing	42
4.1.3.4	Acceptance Criteria	42
4.1.4	Latency Analysis	43
4.1.5	Narrative Quality	44
4.1.6	User Satisfaction	45
4.1.7	Consistency-Checker Performance	46
4.1.8	Image-Synthesis Throughput	46
4.1.9	Character-Extraction Quality	47
4.1.10	Ambient-Music Frequency Profile	48
4.1.11	Question-Budget Ablation	48
4.1.12	Feature Adoption	49
4.1.13	Comparison with Existing Systems	49
4.1.14	Summary of Findings	50
4.2	Discussion	51
4.2.1	Introduction	51
4.2.2	Interpretation of Results	51
4.2.3	Comparison with Previous Research	51
4.2.4	Implications of the Findings	52
4.2.5	Limitations of the Study	52
4.2.6	Recommendations for Future Research	53
5.	Conclusion and Future Scope	54
5.1	Summary of the Study	54
5.2	Main Findings	54
5.3	Contributions to Knowledge	55
5.4	Practical Implications	56
5.5	Future Scope	56
5.6	Research Outcomes	57
5.7	Final Thoughts	57

Bibliography	58
Appendices	63
A. Sample API Exchange	64
B. System Summary and Reproducibility	65

List of Figures

Figure 3.1 Project Gantt chart (August 2025 to May 2026).	19
Figure 3.2 Layered architecture of the proposed system.	21
Figure 3.3 Research workflow followed in this dissertation.	22
Figure 3.4 Distribution of the 17 questions across the four phases.	23
Figure 3.5 Routing of user-added questions from the interface to the LLM prompt.	24
Figure 3.6 Word-count distribution across the five story phases.	25
Figure 3.7 Landing page with the choice of question budget.	27
Figure 3.8 Single-page question form with custom-answer option.	28
Figure 3.9 Settings panel for provider, model, temperature and feature toggles.	30
Figure 3.10 Per-segment image synthesis pipeline.	31
Figure 3.11 Loading state during story and image generation.	32
Figure 3.12 Generated story segment with its scene illustration.	33
Figure 3.13 Narration and ambient-music signal flows.	34
Figure 4.1 Mean latency by LLM provider ($n = 10$ calls per cell).	43
Figure 4.2 Time of one interactive turn, divided by stage.	44
Figure 4.3 Narrative quality by genre (Likert 1–5, $n = 30$ stories).	44
Figure 4.4 Tokens per segment by genre ($n = 25$ segments each).	45
Figure 4.5 User satisfaction by feature (Likert 1–5, $n = 15$ participants).	45
Figure 4.6 Consistency-checker detection and false-positive rates ($n = 50$ segments).	46
Figure 4.7 Image latency against prompt length, and failure rate under parallel requests.	47
Figure 4.8 Character-extraction quality before and after filtering.	47
Figure 4.9 Energy by frequency band for each genre’s ambient track.	48
Figure 4.10 Share of user-study sessions in which each feature was used.	49
Figure 4.11 Comparison with existing systems on seven dimensions.	50

List of Tables

Table 1.1	Measurable performance targets.	4
Table 2.1	Summary of reviewed literature on AI-driven story generation. . . .	14
Table 2.2	Feature-level gap analysis against representative systems.	15
Table 3.1	Risk register and mitigation.	20
Table 3.2	Word-count thresholds of the five story phases.	25
Table 3.3	LLM providers supported by the system.	29
Table 3.4	REST API endpoints.	36
Table 3.5	Technology stack.	37
Table 3.6	Implementation size in lines of code.	38
Table 4.1	Unit-test coverage by module.	41
Table 4.2	API testing results.	42
Table 4.3	Mean Likert scores by question budget ($n = 15$).	49
Table 4.4	Results against the targets set at the start of the work.	50
Table B.1	Summary of the final system.	65

Chapter 1

Introduction

1.1 Background

Stories are one of the oldest ways in which people share knowledge, values and experience. Children learn through them, teachers use them to explain difficult ideas, and whole industries in publishing, film and games are built around them. For most of history a story has been something written by one person and read by another. The reader could interpret it, but could not change it.

Computers have been used to generate stories since the 1970s. Early systems such as TALE-SPIN [Meehan, 1977] used hand-written rules and story grammars to produce short fables in a closed world. Statistical and recurrent neural models later learned patterns from collections of stories, but they struggled to stay coherent for more than a few sentences. The transformer architecture [Vaswani et al., 2017] changed this. Its attention mechanism lets a model relate words that are far apart in a text, and large models trained on very large corpora became able to write long passages that read naturally [Brown et al., 2020; OpenAI, 2023b; Touvron et al., 2023; Jiang et al., 2023].

Modern Large Language Models (LLMs) such as the GPT series [OpenAI, 2023b], LLaMA [Touvron et al., 2023], BERT and T5 [Devlin et al., 2019] and BART [Lewis et al., 2020] can now understand natural-language requests, keep track of context over a conversation and write text with a chosen tone and style. These abilities make it possible to change how stories are made. Instead of giv-

ing the user a finished story, a system can ask the user what they want, listen to the answers and build the story with them. Q&A based story generation follows this idea. The interaction becomes a guided dialogue from which the story grows, and the user becomes a co-author rather than a reader.

1.2 Motivation

Most current AI story tools work on a single prompt. The user writes one request and the model returns one story. This way of working has several practical problems:

- **Preferences must be known in advance.** The user has to describe genre, characters, setting and mood in one go. Many users find this hard, and they often discover what they want only after they see a first draft.
- **Generic output.** A short prompt leaves most decisions to the model, so stories tend to look alike and do not reflect the individual user.
- **No control during the story.** Once generation has started, the user cannot steer the plot, add a character or change direction without starting again.
- **Text only.** Most tools produce plain text. Pictures, narration and music, which make stories more engaging, have to be added by hand with other tools.

Interactive storytelling is useful in many areas. In education, stories that adapt to a learner's interest improve attention and recall. In entertainment and games, players want to shape the outcome through their own choices. In therapy, personalised narratives are used to support reflection and emotional processing. In content creation and marketing, there is demand for personalised stories at scale. Finally, a guided Q&A process makes creative writing accessible to people who have rich ideas but little experience of writing. These needs were the main motivation for this work.

1.3 Problem Statement

Existing story-generation systems face both technical and user-experience limitations:

1. **Limited context.** An LLM can attend only to a fixed number of tokens. As a story grows, early details can be forgotten or contradicted.
2. **Loss of coherence.** When a user adds input at several points, each new input can introduce small inconsistencies in plot, characters or tone, and these build up over time.
3. **Difficulty in reading intent.** Free-text answers are often vague or indirect, and turning them into clear story decisions is not straightforward.
4. **Balance between control and quality.** Giving the user full control can lead to a poorly structured story, while giving too little control reduces personalisation.
5. **Multimodal delays and failures.** Adding images, narration and music introduces extra latency and, with free services, rate-limit errors that the interface must hide.
6. **No reader aids.** Few tools give readers a view of the story structure or of the characters, which makes longer stories hard to follow.

The problem addressed in this dissertation can therefore be stated as follows: *to design and implement a system that collects a user's story preferences through a structured, real-time question-and-answer dialogue, turns them into a coherent and personalised multi-chapter story using a large language model, enriches the story with illustration, narration and ambient music, and lets the user extend the question set with their own questions, while remaining independent of any single model provider and running on ordinary hardware at little or no cost.*

1.4 Objectives of the Study

The specific objectives of this work are:

1. **Structured input.** To design a Q&A framework that captures user preferences through natural conversation, with budgets of 4, 8, 12 and 17 questions ordered across setting, character, theme and plot, a custom-answer option for every question, and a way for users to add their own questions.
2. **Personalised generation.** To build LLM-based story generation that reflects the user’s answers and works with Ollama, Groq, Hugging Face and OpenAI, with the provider switchable at run time.
3. **Narrative coherence.** To keep stories consistent across many turns using a five-phase story-flow model and a consistency checker for character, plot, pacing, setting and tone problems.
4. **Multimodal output.** To add a scene illustration for each segment, voice narration and genre-matched ambient music, all driven by the current segment and the detected genre.
5. **Reader analytics.** To provide a Story Map and a Character Relationship Graph that help readers navigate the generated story.
6. **Evaluation.** To evaluate the system with automatic measurements and a user study covering coherence, creativity, engagement, consistency and personalisation.

The measurable targets set at the start of the work are listed in Table 1.1.

Table 1.1: Measurable performance targets.

Metric	Target	Method
Narrative coherence score	> 85%	Discourse analysis
User-preference alignment	> 80%	Likert vs. stated preference
User satisfaction rating	> 4.0/5	Post-session survey
Context retention accuracy	> 90%	Cross-segment consistency
Response latency (cloud)	< 5 s	End-to-end timing
Session completion rate	> 75%	Sessions completed / started
Image generation success	> 95%	Success on first try or retry

1.5 Research Questions

The work is guided by the following questions:

1. How can a structured Q&A dialogue capture a user's preferences for genre, tone, characters, plot and theme well enough to drive story generation?
2. Which design choices allow an LLM to keep a story coherent over several turns while still taking in new user input?
3. How should the story context be stored and selected so that important details are kept while the story is free to develop?
4. Which measures are suitable for judging an interactive storytelling system on coherence, personalisation, engagement and creativity?
5. How can user control be balanced against the need for a well-structured story?

1.6 Scope of the Work

The scope of this work is defined as follows:

- **Text first.** The story is text, supported by illustration, narration and ambient music. Video generation is outside the scope.
- **English only.** Stories are generated in English. Other languages are left for future work.
- **Fiction genres.** The system supports eight genres: fantasy, science fiction, mystery, romance, thriller, adventure, horror and comedy.
- **Single user.** Each session is used by one person. Multi-user collaborative stories are future work.
- **Short to medium length.** Stories range from about 500 to 5,000 words, usually in two to six chapters.

1.7 Significance of the Study

On the practical side, the study shows that a complete interactive storytelling system with pictures, narration and music can be built entirely from free and

open-source components and run on a single laptop. The free Groq tier gives story segments in under two seconds, so the approach is usable by schools, small studios and individual creators who do not have large budgets.

On the research side, the work introduces the `custom_elements` context channel, a simple pattern that lets end users add their own questions and have the answers reach the model’s prompt without any change to code or database schema. The pattern is not specific to stories and can be used in any LLM application that needs user-extensible personalisation. The study also reports a reproducible evaluation with 50 stories and 15 participants, and reports the limits of its own components openly, for example by labelling the character graph as a co-appearance graph.

1.8 Contributions

The main contributions of this dissertation are:

1. A Q&A framework for interactive storytelling with 17 questions over four narrative dimensions and four selectable budgets.
2. A seven-module backend pipeline that separates context extraction, prompt engineering and story-arc control and does not depend on any one LLM provider.
3. A multimodal layer with sentence-scored text-to-image prompts, browser-based narration and genre-matched ambient music, together with a serial request queue that prevents rate-limit failures.
4. Two D3-based reader aids: a Story Map of the chapters and a Character Relationship Graph.
5. An end-user authoring layer that accepts custom answers and user-added questions and passes them to the model through the `custom_elements` channel.
6. An evaluation covering latency, narrative quality, user satisfaction, consistency checking, image throughput and a question-budget ablation.

1.9 Structure of the Report

The rest of this report is organised as follows.

Chapter 2 reviews the literature on automated story generation. It covers rule-based and neural approaches, interactive storytelling systems and multimodal co-creation, and ends with the research gaps that this work addresses.

Chapter 3 describes the proposed methodology and system design. It covers the feasibility study, the project plan, the system architecture, the Q&A framework, the algorithms, the seven backend modules, the multimodal pipelines, the implementation and the evaluation method.

Chapter 4 presents the testing and experimental results and discusses them. It covers latency, narrative quality, user satisfaction, consistency checking, image throughput, character extraction, the question-budget ablation and a comparison with existing systems.

Chapter 5 concludes the report, summarises the main findings, lists the limitations and outlines directions for future work.

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews earlier work that is relevant to Q&A based interactive storytelling. It begins with an overview of how automated story generation has developed, and then explains the main theoretical ideas on which the proposed system rests: narrative structure, transformer language models, prompt engineering, text-to-image diffusion and speech synthesis. After that it reviews previous research on controllable, interactive and multimodal story generation, summarises the reviewed work in a table, and identifies the research gaps that this dissertation addresses.

2.2 Overview of the Topic

2.2.1 Evolution of Automated Story Generation

Automated story generation has passed through several stages. The first systems, built in the 1970s and 1980s, were rule based. TALE-SPIN [Meehan, 1977] produced short animal fables by simulating characters who had goals and planned actions to reach them. UNIVERSE [Lebowitz, 1985] treated storytelling as planning at the level of the author, and MEXICA [Pérez y Pérez and Sharples, 2001] modelled creative writing as a cycle of engagement and reflection. These systems showed that a computer could produce a coherent plot, but they depended on hand-written knowledge and worked only in narrow domains.

The statistical era brought models that learned from collections of existing stories. They were more flexible but found it hard to stay coherent over long passages. Recurrent neural networks such as LSTMs and GRUs captured sequences better, yet their memory of earlier text faded as a story grew.

The transformer [Vaswani et al., 2017] replaced recurrence with attention, which lets every word look directly at every other word in the context. Large pretrained models built on it, such as GPT-2 [Radford et al., 2019], GPT-3 [Brown et al., 2020] and GPT-4 [OpenAI, 2023b], showed that a model trained on enough text could write long, fluent and creative stories. Open models such as LLaMA [Touvron et al., 2023] and Mistral [Jiang et al., 2023] later made this ability available for local use. Surveys of the field [Zhao et al., 2024; Naveed et al., 2024; Wang et al., 2025] note that the main open problems have moved from fluency to control, coherence over long texts and meaningful user involvement.

2.2.2 Interactive Storytelling

Interactive storytelling lets the user influence the story while it is being told. Early work on believable agents and story adaptation [Riedl and Stern, 2006] used planning to change a story in response to player actions while keeping it coherent. With LLMs, interaction has become much more open. AI Dungeon [Walton, 2019] lets the player type any action and have the model continue the story. Such free-form input gives the user great freedom, but it also puts the whole burden of direction on the user and often leads to stories that drift.

2.2.3 Multimodal Storytelling

Stories are more engaging when text is combined with images and sound. Recent surveys [Jia et al., 2024; Liu et al., 2025] describe a shift from co-writing towards co-imagining, in which people and AI systems create text, pictures and audio together. This is now practical because good text-to-image and text-to-speech models are freely available.

2.3 Theoretical Framework

2.3.1 Narrative Structure

Classical narratology describes how stories are organised [Bal, 2017]. One of the most widely used models is Freytag’s pyramid [Freytag, 1900], which divides a plot into five parts: exposition (introduction), rising action, climax, falling action and resolution. The proposed system uses these five parts as phases of a finite-state machine, so that a generated story follows a recognisable dramatic arc instead of wandering.

2.3.2 Transformer Language Models

A transformer maps a sequence of tokens to a sequence of vectors through stacked layers of self-attention and feed-forward networks [Vaswani et al., 2017]. In self-attention, each token forms a query Q , a key K and a value V , and the output is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (2.1)$$

where d_k is the size of the key vectors. Decoder-only models such as GPT and LLaMA generate text one token at a time, each time predicting the next token from all the previous ones. Encoder models such as BERT [Devlin et al., 2019] and encoder-decoder models such as BART [Lewis et al., 2020] and T5 [Raffel et al., 2020] are more often used for understanding and transformation tasks. Instruction tuning with human feedback [Ouyang et al., 2022] makes models follow requests closely, which is essential when a system needs the model to respect specific story constraints. The Hugging Face Transformers library [Wolf et al., 2020] and local runtimes such as Ollama [Ollama Team, 2024] make these models easy to use in applications.

A key practical limit is the *context window*, the maximum number of tokens the model can attend to. Everything the model should remember about the story, including the user’s answers and the earlier segments, has to fit inside it. This is

why the proposed system keeps a structured summary of the user’s preferences and passes only the most recent segments to the model.

2.3.3 Prompt Engineering

A prompt is the text given to a language model to steer its output. Prompt engineering is the practice of designing prompts so that the model produces the desired kind of answer [Liu et al., 2023b; Sahoo et al., 2024]. Useful techniques include giving the model a role in a system message, stating constraints explicitly, providing examples (few-shot prompting) [Brown et al., 2020], writing the prompt as a small program with clear sections [Reynolds and McDonell, 2021], and asking the model to reason step by step [Wei et al., 2022]. In the proposed system, each story phase has its own prompt template, and the user’s answers are inserted into it as structured fields.

2.3.4 Text-to-Image Diffusion

Diffusion models generate an image by starting from random noise and removing it step by step, guided by a text description. Latent diffusion [Rombach et al., 2022] performs this process in a compressed latent space, which makes it fast enough for everyday hardware. Later models such as DALL·E 2 [Ramesh et al., 2022], DALL·E 3 [OpenAI, 2023a], SDXL [Podell et al., 2024] and FLUX.1 [Black Forest Labs, 2024] produce high-quality images from natural-language prompts. Pollinations.ai [Pollinations.ai, 2024] provides free access to such models through a simple URL, which the proposed system uses for scene illustrations. The quality of the image depends strongly on the prompt, so choosing a short, visual description from the story text is important.

2.3.5 Speech Synthesis and Ambient Music

Text-to-speech (TTS) converts written text into spoken audio. Browsers provide a built-in TTS engine through the W3C Web Speech API [W3C Speech API Community Group, 2024], which needs no server or key. Neural TTS models such as XTTS [Casanova et al., 2024] give more natural voices and can imitate a speaker, at a higher computing cost. Background music that matches the mood

of a scene is common in games, and procedural music for interactive narratives has been surveyed by López-Rincón et al. [López-Rincón et al., 2024]. The proposed system uses royalty-free algorithmic tracks from SoundHelix [Brichta, 2024], chosen according to the story genre.

2.4 Previous Research

2.4.1 Controllable and Planned Story Generation

Several studies try to give structure or control to LLM-based stories. Farrell et al. [Farrell et al., 2024] use an LLM as a heuristic guide for a classical narrative-planning search. The plans are well structured, but the user takes no part once generation starts, so personalisation is limited to the initial settings. Huang et al. [Huang et al., 2023] add an external knowledge graph to a GPT-based generator to improve factual consistency and world-building, but the user is a passive reader with no way to express preferences during the process. Tambwekar et al. [Tambwekar et al., 2023] use reinforcement learning to control style and emotion according to constraints given by the user. The control is set at the start and cannot be changed as the story develops.

Li et al. [Li et al., 2023] guide GPT-3 with hierarchical prompts built from a plot outline. Ammanabrolu et al. [Ammanabrolu et al., 2022] use hierarchical neural planning to give long stories a clear structure. Li et al. [Li et al., 2022] add a discourse-aware planner to improve coherence in long-form stories, and Mao et al. [Mao et al., 2022] focus on keeping character personalities consistent. Zhang et al. [Zhang et al., 2024] use instruction-tuned LLMs to generate stories from a single detailed prompt. These works improve structure and consistency, but they follow a linear process without Q&A-driven branching.

2.4.2 Interactive and Co-Creative Systems

Peng et al. [Peng et al., 2023] turn dialogue-based user input into stories with a BART model, which is closer to interactive use, although the dialogue is not structured around story dimensions. Dramatron [Mirowski et al., 2023] builds

scripts hierarchically, from a log-line to scenes to dialogue, and improves coherence for long texts. TaleBrush [Chung et al., 2022] lets writers sketch the fortune of a character as a line and have the model follow it. Wordcraft [Yuan et al., 2022] is an editor in which writers work with an LLM on their own stories. User studies of such tools [Liu et al., 2023a; Yuan et al., 2022] found that guided, scaffolded interfaces help novice users even when the underlying model is the same, because users often do not know how to phrase what they want.

2.4.3 Multimodal Story Generation

Schuhmann et al. [Schuhmann et al., 2024] combine diffusion models with LLM co-pilots to generate illustrated stories, and Bouhuis et al. [Bouhuis et al., 2024] study cross-modal coherence between the text and its pictures. Both report higher reader engagement when images match the current state of the story. These are research prototypes, however, and they do not address the practical rate limits of free image services. Music for interactive narratives is mostly studied in games [López-Rincón et al., 2024; Garg and Rajan, 2024] and rarely combined with generated text.

2.4.4 Evaluation of Generated Stories

Evaluating stories is difficult because there is no single correct output. Wu et al. [Wu et al., 2024] and Lin et al. [Lin et al., 2024] propose multi-dimensional evaluation frameworks that combine automatic scores with LLM-based judgement of coherence, creativity and engagement. Human ratings on Likert scales remain the most common reference. The present work follows this practice and combines automatic measurements with a user study.

2.4.5 Summary of Reviewed Literature

Table 2.1 summarises representative works and the gap each leaves with respect to interactive, Q&A-driven personalisation.

Table 2.1: Summary of reviewed literature on AI-driven story generation.

#	Year	Authors	Method	Gap
1	2024	Farrell et al. [Farrell et al., 2024]	LLM-guided narrative planning	No user interaction
2	2023	Huang et al. [Huang et al., 2023]	GPT with knowledge graph	Limited personalisation
3	2023	Tambwekar et al. [Tambwekar et al., 2023]	RL-based style control	Not interactive
4	2023	Li et al. [Li et al., 2023]	Hierarchical prompting of GPT-3	No Q&A component
5	2022	Ammanabrolu et al. [Ammanabrolu et al., 2022]	Hierarchical transformer	No Q&A branching
6	2022	Mao et al. [Mao et al., 2022]	Character-centric transformer	No Q&A personalisation
7	2024	Zhang et al. [Zhang et al., 2024]	Instruction-tuned LLM	Single-shot prompt
8	2023	Peng et al. [Peng et al., 2023]	BART dialogue-to-story	No structured Q&A
9	2022	Li et al. [Li et al., 2022]	Discourse planner with LLM	Not personalised
10	2024	Wu et al. [Wu et al., 2024]	LLM-based story metrics	Evaluation only
11	2023	Mirowski et al. [Mirowski et al., 2023]	Hierarchical script writing	No multimodal output
12	2024	Schuhmann et al. [Schuhmann et al., 2024]	Diffusion with LLM co-pilot	Prototype, no Q&A

2.5 Research Gaps

The review shows the following gaps:

1. **No structured Q&A interaction.** Planning-based and knowledge-based systems [Farrell et al., 2024; Huang et al., 2023] do not collect preferences through a real-time dialogue, and user answers do not shape later parts of the story.
2. **Control fixed at the start.** Controllable systems [Tambwekar et al., 2023;

[Li et al., 2023](#)] let the user set style or plot only once, with no refinement as the story develops.

3. **Linear structure.** Hierarchical approaches [[Ammanabrolu et al., 2022](#)] improve structure but follow a linear path without user decision points. Personalisation, where present, depends on templates rather than live answers.
4. **No integrated multimodal output.** None of the reviewed systems combines illustration, narration and ambient music driven by the current segment and genre in a deployable application, and none deals with the rate limits of free image services.
5. **No end-user authoring of questions.** No existing system lets the user add entirely new questions next to the system’s own questions and have them influence the story.

Table 2.2 compares the proposed system with three well-known tools on these features.

Table 2.2: Feature-level gap analysis against representative systems.

Feature	Proposed	AI Dungeon	Dramatron	NovelAI
User input	Structured Q&A	Free-form	Hierarchical	Free-form
Structure management	5-phase	None	Scene-based	Limited
Consistency checking	Active	None	Log-line	Memory
LLM providers	4 (free + paid)	OpenAI	PaLM	Custom
Local deployment	Yes	No	No	No
Image synthesis	Yes	No	No	Partial
TTS narration	Yes	No	No	No
Ambient music	Genre-based	No	No	No
Custom questions	Yes	No	No	No
Open source	Yes	Partial	Yes	No

Taken together, these gaps point to one central issue: existing systems do not treat the user as an active co-creator. The ability to generate good text exists, but a framework that uses it through meaningful, multimodal and user-extensible

interaction is still missing.

2.6 Summary

This chapter traced automated story generation from rule-based systems to large language models, explained the theory behind the components used in this work, and reviewed recent research on controllable, interactive and multimodal storytelling. The components needed for the proposed system (LLM generation, conversational interaction, image and speech synthesis) exist separately, but no reviewed system brings them together in one accessible, provider-independent platform. The next chapter describes the design of such a system.

Chapter 3

Proposed Methodology and System Design

3.1 Introduction

This chapter describes the design and implementation of the Q&A based interactive storytelling system developed in this dissertation. It starts with the feasibility study and the project plan, then gives an overview of the system and the research workflow. It then explains each part in turn: the question framework, the way answers are turned into a story context and a prompt, the five-phase story model, the seven backend modules, the multimodal pipelines and the reader analytics. The implementation, the hardware and software used, and the evaluation method are described at the end of the chapter.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every part of the system can be built with mature and freely available technology. LLM inference is possible on an ordinary laptop through Ollama [[Ollama Team, 2024](#)] (16 GB RAM and no dedicated GPU for 7B-class models) and in the cloud through the free Groq tier, which allows 14,400 requests per day. Text-to-image synthesis is available without an API key through Pollinations.ai [[Pollinations.ai, 2024](#)]. Voice synthesis is built into all major browsers through the Web Speech API [[W3C Speech API Community Group, 2024](#)], and ambient music can be streamed from the SoundHelix CDN [[Brichta, 2024](#)]. The back-

end can be written in Python with FastAPI [Ramírez et al., 2024], SQLite and Pydantic [Colvin, 2024], and the frontend in plain HTML, CSS and JavaScript with D3.js [Bostock et al., 2011] for charts.

3.2.2 Economic Feasibility

The system can be developed and run at almost no cost. Development used Ollama and the free Groq tier; images are generated free of charge through Pollinations; and audio streaming uses very little bandwidth from a public CDN. The recurring cost during the user study, with 15 participants over two weeks, was zero. All development was done on one laptop, so no external funding was required.

3.2.3 Operational Feasibility

The Q&A protocol follows the way a human author collects a brief from a client, through a short interview about setting, characters, theme and plot, so it is familiar to users. The question budget (4 to 17 questions) lets each user choose how much effort to spend. All steps take place on a single page, and the settings and admin panels expose every option without any change to code.

3.2.4 Legal and Ethical Feasibility

All third-party services used (Pollinations, SoundHelix, Groq, Hugging Face and OpenAI) allow non-commercial research use, and SoundHelix explicitly allows direct linking. No personally identifiable data is stored permanently: session data is kept in memory and in a local SQLite file. The user study was carried out with informed consent.

3.3 Project Planning and Scheduling

The project ran from August 2025 to May 2026. The work was broken down into eleven activities in five phases:

1. **Planning and design** (weeks 1–12): literature survey, requirement analysis with the guide, and system design covering the architecture, module

boundaries, data flow, REST interface and database schema.

2. **Implementation** (weeks 10–23): the seven backend modules, the LLM provider layer with Ollama, Groq, Hugging Face and OpenAI clients, and the frontend with the landing page, question form, story view, admin panel and settings panel.
3. **Multimodal features** (weeks 20–26): the image pipeline with the prompt extractor and request queue, TTS narration, ambient music, and the Story Map and Character Graph.
4. **Evaluation** (weeks 25–29): automatic generation of 50 stories, the user study with 15 participants, and the consistency-checker evaluation.
5. **Documentation** (weeks 28–36): the dissertation report, a research paper, and the final review and presentation.

The timeline is shown in Figure 3.1. Some activities overlap on purpose; for example, frontend work ran alongside backend work, and multimodal integration overlapped with the end of frontend development.

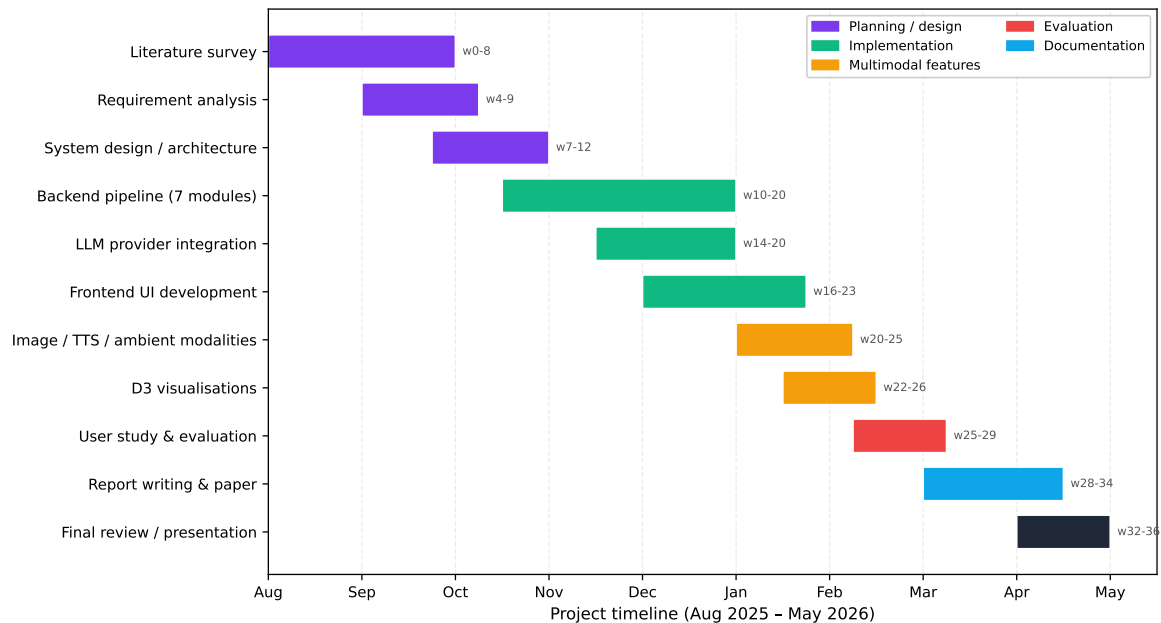


Figure 3.1: Project Gantt chart (August 2025 to May 2026).

The main risks identified during planning and the way each was handled are listed in Table 3.1.

Table 3.1: Risk register and mitigation.

Risk	Likelihood / Impact	Mitigation
Rate limits of the free image API	High / High	Serial request queue with 1.2 s spacing and a 35 s watchdog
LLM provider downtime	Medium / Medium	Provider-independent layer that can be switched through <code>/api/settings</code>
Loss of narrative coherence	Medium / High	Sliding-window context and a consistency checker
Browser blocks audio autoplay	High / Low	Start audio only after a real user click
Delay in the user study	Medium / Medium	Rolling sessions over two weeks, remote participation allowed
Cross-origin errors for CDN audio	Medium / Low	Plain <code>HTMLAudio</code> without the <code>crossOrigin</code> attribute

3.4 System Overview

The system is organised in layers, shown in Figure 3.2. The *presentation layer* is a single-page web application that shows the questions, the story, the images and the analytics, and plays narration and music. The *application layer* is a FastAPI server that exposes the REST endpoints and manages sessions. The *modular core* contains the seven backend modules that turn answers into a story. At the bottom, a pluggable *LLM provider layer* connects the core to Ollama, Groq, Hugging Face or OpenAI. Each layer depends only on the layer below it, so a provider can be changed or a module improved without touching the rest.

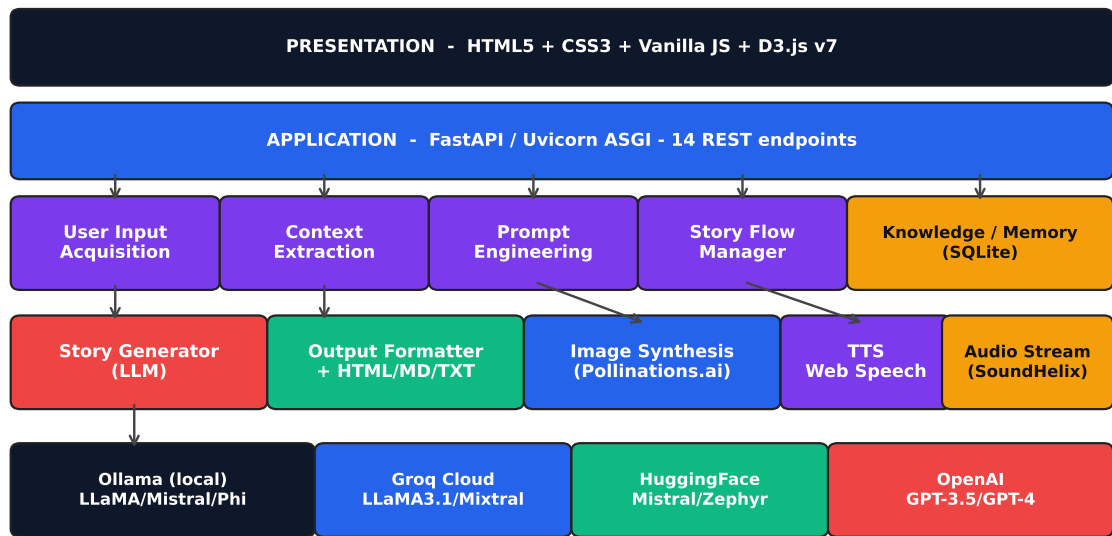


Figure 3.2: Layered architecture of the proposed system.

3.5 Research Design

The research followed the workflow shown in Figure 3.3. Requirements were gathered from the literature and from discussion with the guide, the system was designed and built module by module, and the multimodal features were added on top of a working text pipeline. The complete system was then tested and evaluated with automatic measurements and a user study. Problems found in testing were fixed and the affected tests were run again.

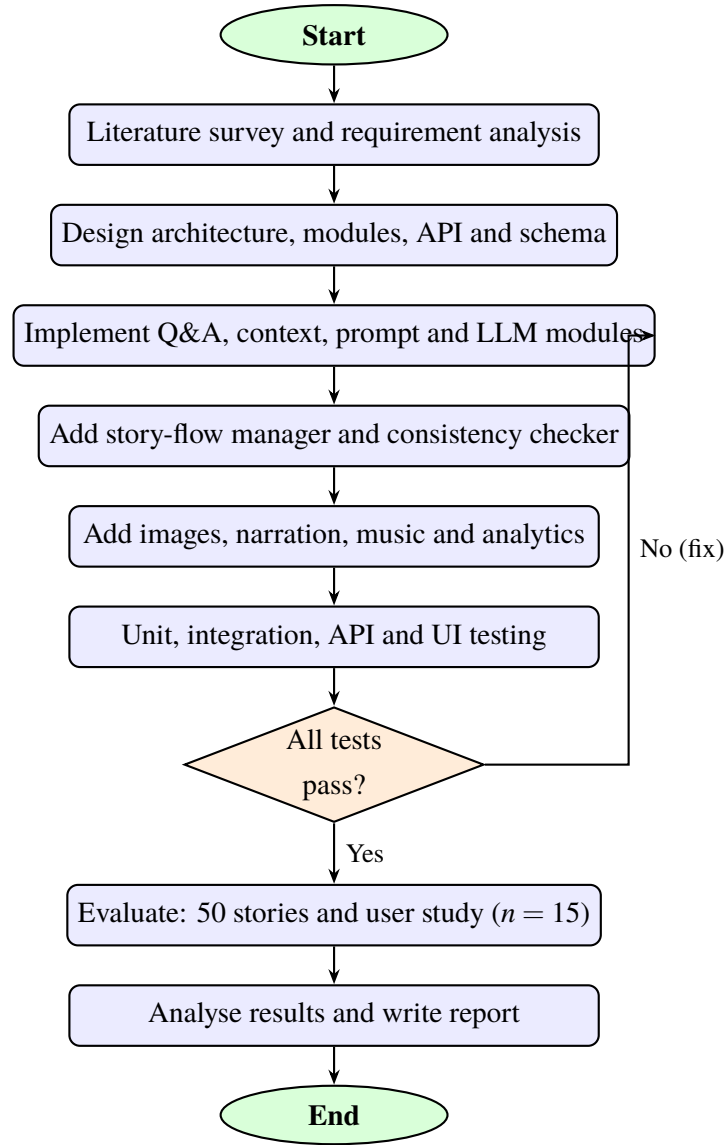


Figure 3.3: Research workflow followed in this dissertation.

3.6 Question-and-Answer Framework

3.6.1 Question Pool

The elicitation protocol has seventeen questions divided among four narrative phases, as shown in Figure 3.4:

- **Setting:** the kind of world, the time period and the atmosphere of the story.
- **Character:** the protagonist’s name, personality and goal, the antagonist, and supporting characters.
- **Theme:** the genre, the tone and the central message.

- **Plot:** the type of conflict, the plot style, the pacing and the kind of ending.

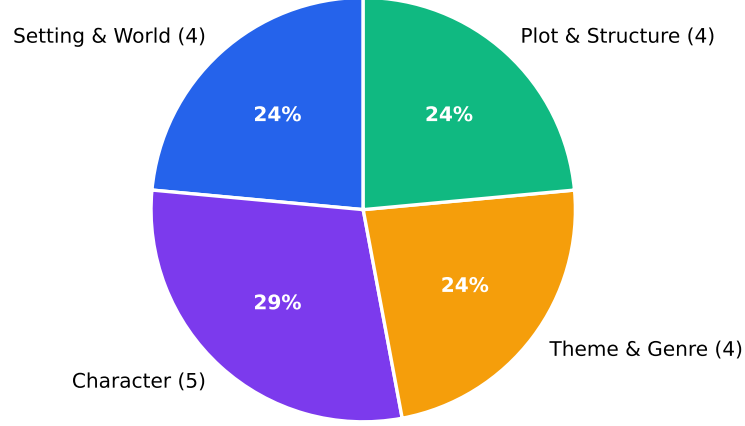


Figure 3.4: Distribution of the 17 questions across the four phases.

The user selects a budget of 4 (Quick), 8 (Balanced), 12 (Detailed) or 17 (Full) questions. The system always presents the first N questions in phase order, so even the smallest budget touches all four dimensions. Every question offers multiple-choice options and a “Write my own” option for a free-text answer. In addition, the user may add entirely new questions with their own answers.

3.6.2 Elicitation Protocol

Let $\mathcal{Q} = \{q_1, \dots, q_{17}\}$ be the ordered question pool. The user selects a budget $N \in \{4, 8, 12, 17\}$ and gives an answer a_i to each question q_i with $i \leq N$. The user may also add M new question-answer pairs $(\tilde{q}_j, \tilde{a}_j)$, which receive the identifiers `custom_q_j`. The complete response set is

$$R = \{(q_i, a_i)\}_{i=1}^N \cup \{(\text{custom_q_}j, \tilde{a}_j)\}_{j=1}^M, \quad (3.1)$$

and it is sent to the context-extraction module as a JSON dictionary.

3.6.3 Context Projection and Prompt Construction

For each pair (q_i, a_i) the context-extraction module applies a fixed projection π_i that writes the answer into a typed field of a `StoryContext` object, for example

setting_type, antagonist or plot_style. Answers whose identifiers are not known to the system, including all user-added questions, are stored word for word in a dictionary D called custom_elements. The final prompt P is built as

$$P = T(\mathbf{c}) \oplus \sigma(D), \quad (3.2)$$

where T is the prompt template for the current story phase, $\mathbf{c}$ is the structured context, σ turns the dictionary D into a list of natural-language bullet points, and $\oplus$ joins the two strings. This design means that a new question added by the user reaches the model without any change to code or to the database schema. Figure 3.5 shows this routing.

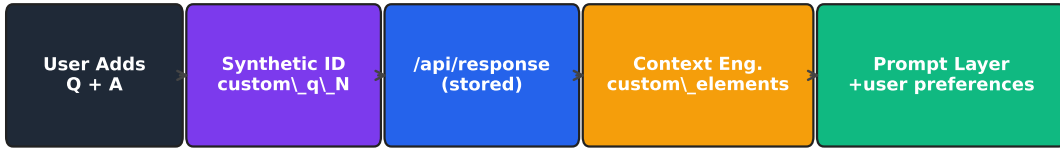


Figure 3.5: Routing of user-added questions from the interface to the LLM prompt.

3.7 Five-Phase Story Progression

The story-flow manager is a finite-state machine based on Freytag’s pyramid [Freytag, 1900]. The story moves through five phases according to its total word count, as listed in Table 3.2. The current phase selects the prompt template: the introduction template asks the model to set the scene and introduce the characters, the climax template asks for the main confrontation, and the resolution template asks the model to close open plot threads. Figure 3.6 shows the observed word counts per phase; the rising action takes the largest share of the text (about 820 words), which agrees with narrative theory.

Table 3.2: Word-count thresholds of the five story phases.

Phase	Word range	Purpose of the prompt
Introduction	0–400	Establish setting and characters
Rising action	400–1200	Develop conflict and complications
Climax	1200–1800	Main confrontation or turning point
Falling action	1800–2400	Consequences of the climax
Resolution	> 2400	Resolve open threads and conclude

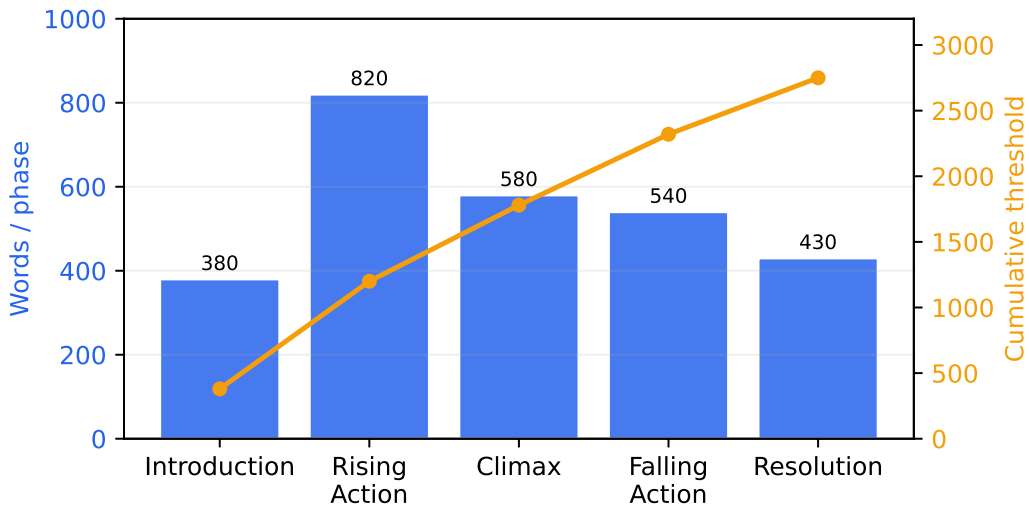


Figure 3.6: Word-count distribution across the five story phases.

3.8 Generation Loop

Each turn of the story follows Algorithm 1. The model sees the structured context and only the last k segments of the story, which keeps the prompt within the context window while preserving the most recent events. Image generation starts in parallel with post-processing so that the picture is often ready when the user finishes reading.

Algorithm 1 Story generation loop for one turn

Require: session s , structured context $\mathbf{c}$, custom elements D , user choice u (empty on the first turn)

Ensure: formatted segment and choices for the next turn

- 1: $H \leftarrow$ last k segments of session s $\triangleright$ sliding window, $k = 4$
 - 2: $\phi \leftarrow$ current phase from total word count
 - 3: $P \leftarrow T_\phi(\mathbf{c}, H, u) \oplus \sigma(D)$
 - 4: $y \leftarrow \text{LLM}(P)$ $\triangleright$ active provider
 - 5: $y \leftarrow \text{FORMAT}(y)$ $\triangleright$ clean text, style dialogue
 - 6: update character and plot-thread trackers with y
 - 7: run consistency checks on y against H and $\mathbf{c}$
 - 8: start image request for y asynchronously
 - 9: save y to the database; update word count
 - 10: **return** y and generated choices
-

3.9 Backend Modules

The backend has seven modules with clear responsibilities and typed Pydantic interfaces.

3.9.1 Module 1: User Input Acquisition

This module holds the question pool and serves it through two endpoints: `/api/questions/all` returns all questions grouped by phase, and `/api/question/next` supports an older one-question-at-a-time mode. Answers are received through POST `/api/response` and validated. The landing page (Figure 3.7) lets the user choose a budget, and the question form (Figure 3.8) shows each question with its options and a dashed “Write my own” chip.

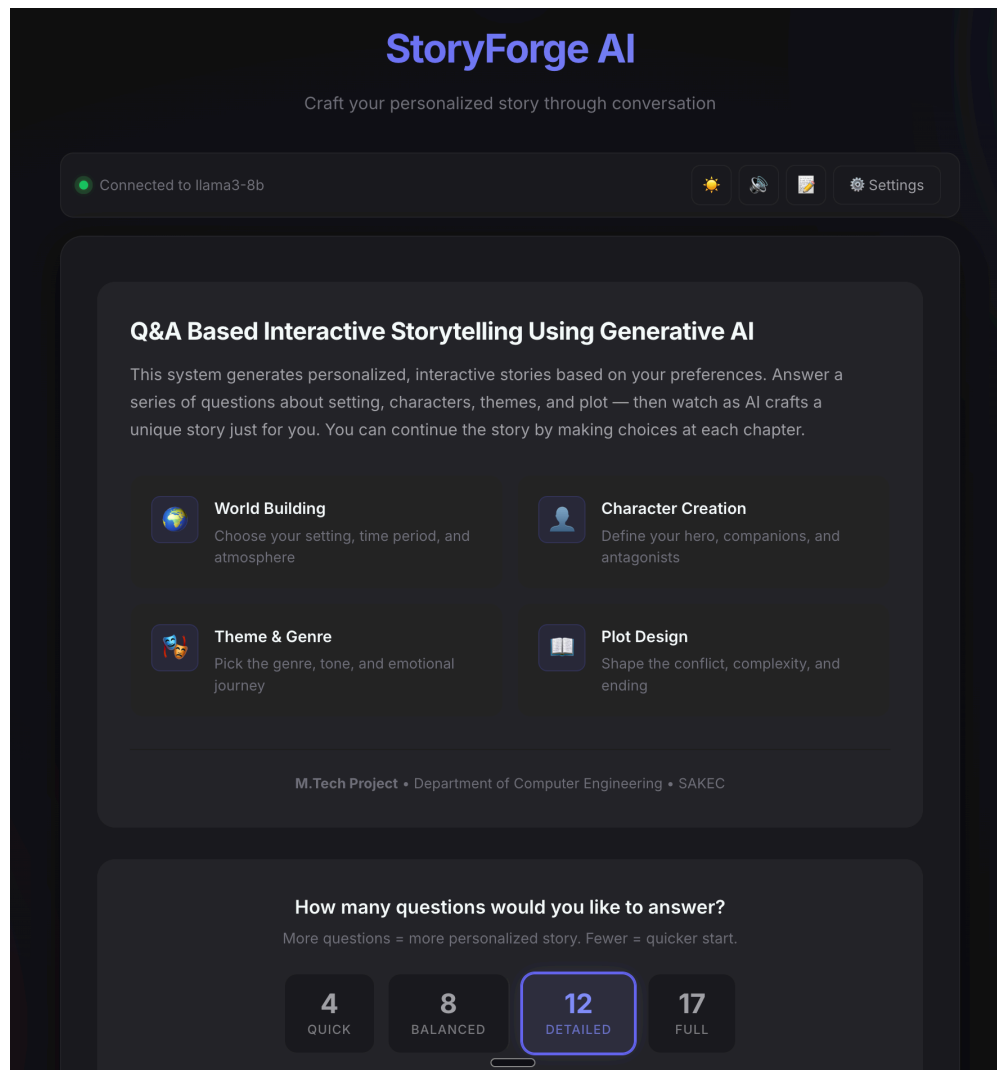


Figure 3.7: Landing page with the choice of question budget.

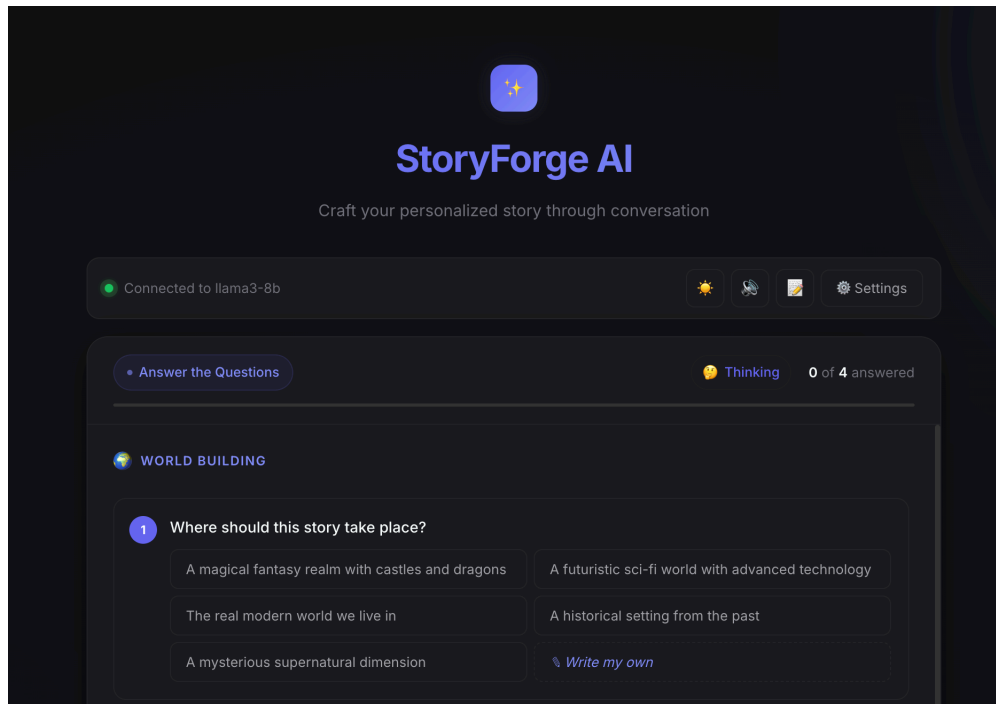


Figure 3.8: Single-page question form with custom-answer option.

3.9.2 Module 2: Context Extraction Engine

This module fills the `StoryContext` object. It works in two passes: first it classifies answers with keyword dictionaries (six genres, five setting types and five conflict types), and then it extracts proper names with regular expressions. Identifiers that begin with `custom_` are routed into `custom_elements`:

```
elif question_id.startswith("custom_"):
    self.context.custom_elements[question_id] = answer
```

3.9.3 Module 3: Knowledge and Memory

This module stores sessions, answers, segments and preferences in SQLite, keeps an in-memory LRU cache for active sessions, and provides the sliding-window helper used in Algorithm 1:

```
def get_recent_context(session_id: str, k: int = 4) -> str:
    """Return the concatenated text of the last k segments."""
```

3.9.4 Module 4: Prompt Engineering

This module holds six prompt families: the system prompt, the initial-segment prompt, the continuation prompt, the climax prompt, the resolution prompt and the choice-generation prompt. Each is filled with fields from `StoryContext`, and genre-specific guidance is added (for example, magic systems for fantasy, clue placement and red herrings for mystery, and emotional pacing for romance). User-added preferences are appended at the end:

```
if context.custom_elements:
    extras = "\n".join(f"- {v}" for v in context.custom_elements.values() if
        v)
    if extras:
        prompt += "\n\nAdditional user preferences to honour:\n" + extras
```

3.9.5 Module 5: Story Generator

The story generator defines an abstract class `BaseLLMClient` with four concrete clients, compared in Table 3.3. A factory creates the client named in the configuration. The settings panel (Figure 3.9) lets a non-technical user choose the provider, model, temperature and features; a change is sent to `POST /api/settings`, which replaces the client without restarting the server.

Table 3.3: LLM providers supported by the system.

Provider	Models	Cost	Latency
Ollama	LLaMA 3.2, Mistral, Phi	Free (local)	8–15 s
Groq	LLaMA 3.1 8B/70B, Mixtral	Free, 14,400/day	1–3 s
Hugging Face	Mistral-7B, Zephyr, Falcon	Free (rate-limited)	6–10 s
OpenAI	GPT-3.5-turbo, GPT-4	Paid per token	2–4 s

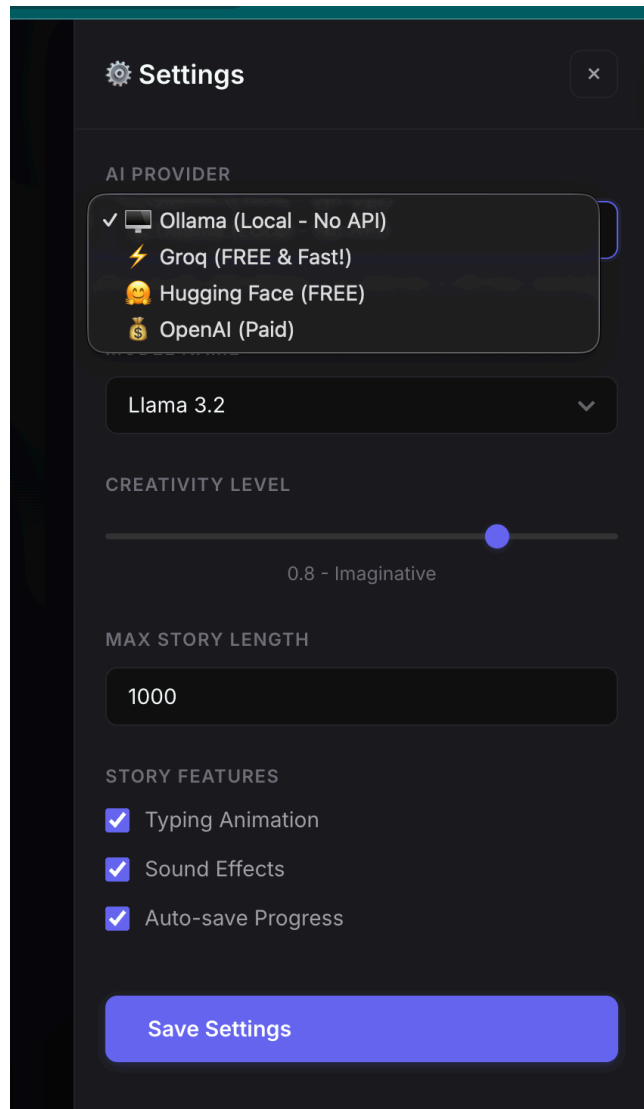


Figure 3.9: Settings panel for provider, model, temperature and feature toggles.

3.9.6 Module 6: Story Flow Manager

This module implements the five-phase state machine of Section 3.7. A `CharacterTracker` records each character’s mentions and traits, and a `PlotThreadTracker` records open plot threads. The consistency checker uses both to flag five kinds of problem: character contradictions, abandoned plot threads, pacing problems, setting inconsistencies and tone shifts.

3.9.7 Module 7: Output Formatter

The formatter cleans the model’s output by collapsing extra whitespace and fixing quote and dash characters, splits text at sentence boundaries, wraps dialogue

in styled HTML spans, and exports the full story as plain text, Markdown or HTML.

3.10 Multimodal Pipelines

3.10.1 Scene Illustration

For each segment, a short image prompt is built from the most visual sentence. Dialogue in quotation marks and sentences built around thought verbs (*thought*, *remembered*, *wondered*) are removed first, because they describe things that cannot be drawn. Each remaining sentence s is scored as

$$\text{score}(s) = 2 \cdot \mathbf{1}_V(s) - 2 \cdot \mathbf{1}_T(s) + \min(3, |\text{ProperNouns}(s)|), \quad (3.3)$$

where $\mathbf{1}_V(s)$ is 1 if the sentence contains a visual verb (*stood*, *loomed*, *soared*, *glowed*, *towered*, and so on) and $\mathbf{1}_T(s)$ is 1 if it contains a thought verb. The best sentence is combined with the genre, the current mood and a fixed style prefix, and the prompt is limited to 200 characters. The pipeline is shown in Figure 3.10.

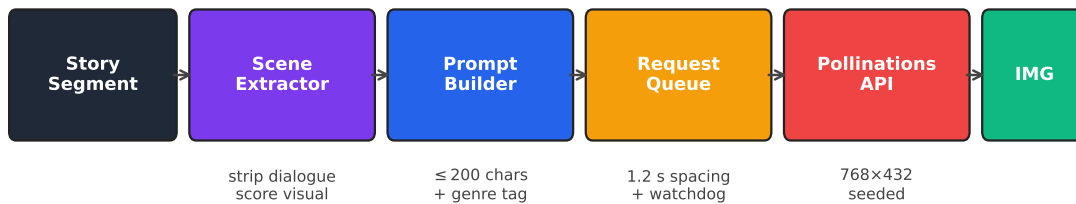


Figure 3.10: Per-segment image synthesis pipeline.

The prompt is sent to Pollinations.ai as a URL:

```

imageUrlFor(segmentText, seed, opts={}) {
  const prompt = this.buildImagePrompt(segmentText, opts);
  const params = new URLSearchParams({
    width: '768', height: '432', nologo: 'true',
    seed: String(seed)
  });
}

```

```
return `https://image.pollinations.ai/prompt/${encodeURIComponent(prompt)}?${params}`;  
}
```

Pollinations is served through Cloudflare, which rejects bursts of parallel requests. All image requests therefore pass through a serial queue that sends one request at a time, waits 1.2 s between requests, and abandons a request after a 35 s watchdog so that a hung request cannot block the queue. Figure 3.11 shows the loading state while the LLM call and the image request run, and Figure 3.12 shows a finished segment with its illustration.

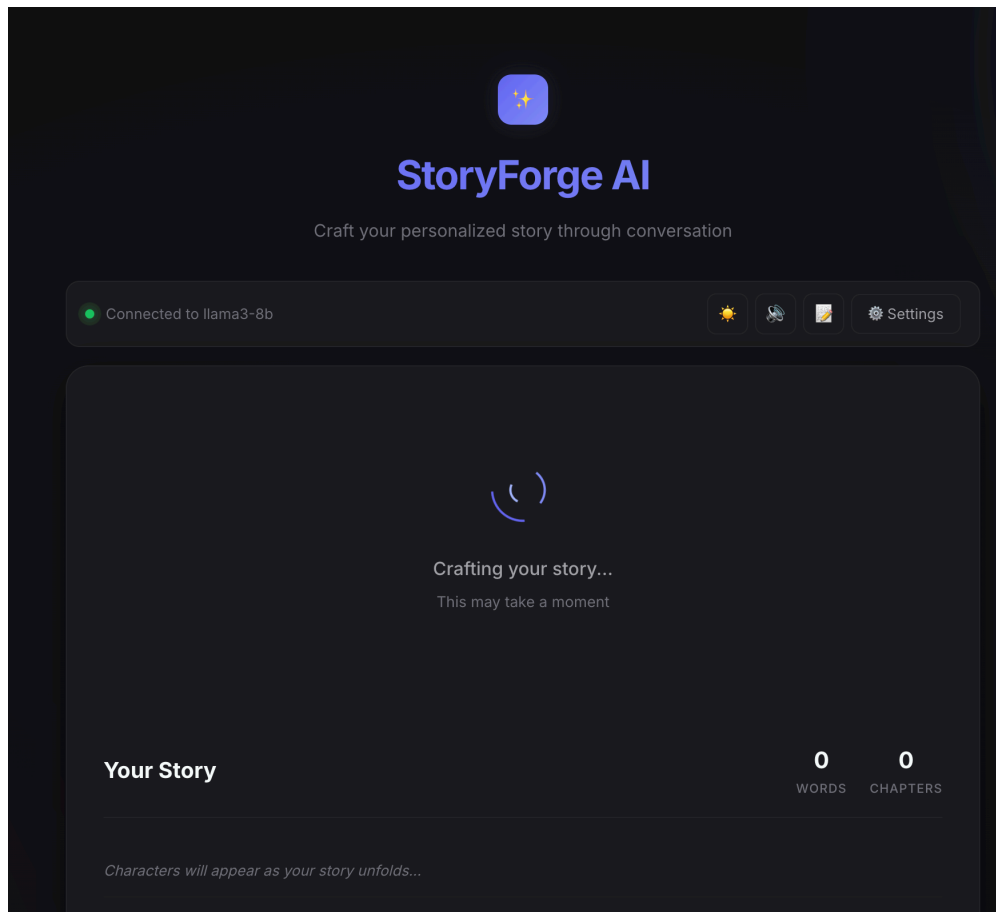


Figure 3.11: Loading state during story and image generation.

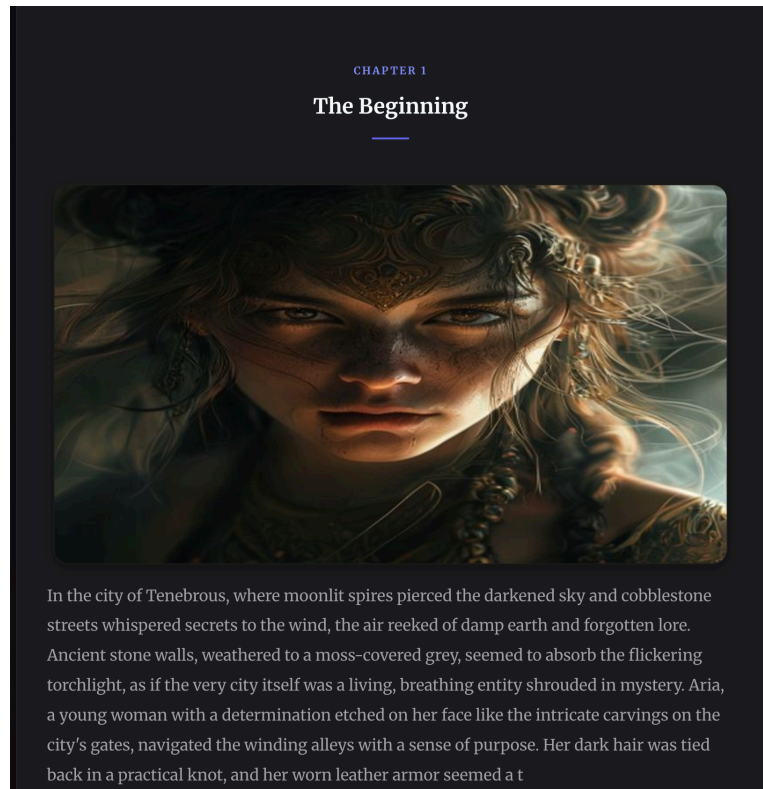


Figure 3.12: Generated story segment with its scene illustration.

3.10.2 Narration and Ambient Music

The two audio pipelines are shown in Figure 3.13. Narration uses the browser's `window.speechSynthesis` interface. One utterance is queued per segment, with pause, resume and stop controls, and the voice is the one provided by the operating system. Ambient music maps each of eight genres to a SoundHelix track, played through `HTMLAudio` in a loop with a 2.4 s fade-in and a 0.8 s fade-out. The `crossOrigin` attribute is not set, because the CDN does not return CORS headers; setting it caused loading to fail in an early version.

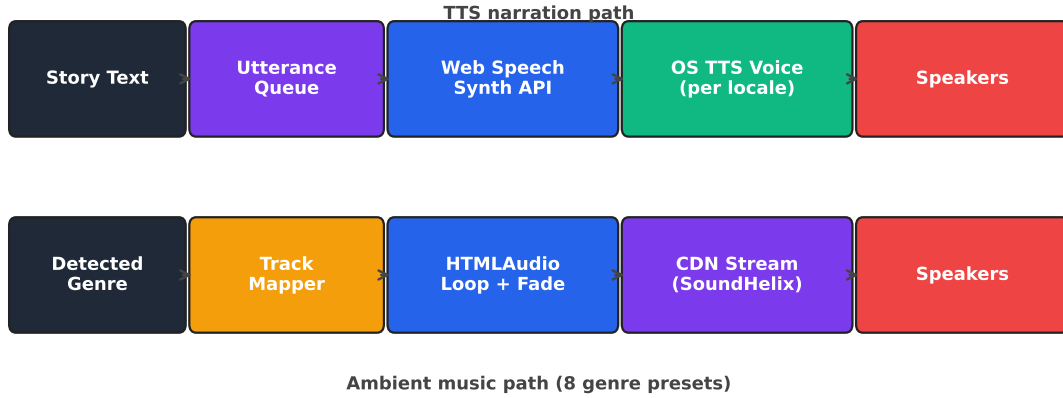


Figure 3.13: Narration and ambient-music signal flows.

3.11 Reader Analytics

3.11.1 Story Map

The Story Map arranges the chapters as a vertical tree using `d3.tree()`. Each node shows an 80-character preview of its chapter and the user choice that led to it, so the reader can see how the story branched.

3.11.2 Character Relationship Graph

Capitalised words are taken from the segment text and filtered with an extended stop-word list. Only names that appear at least twice are kept. For every pair of kept names, the edge weight is the number of segments in which both appear, and the node radius grows with the number of mentions. The graph is drawn with a D3 force layout (link distance 110, charge -220 , collision radius 34 pixels). Because the edges are based only on co-appearance, the modal states clearly that the graph shows which characters appear together and not the type of relationship between them.

3.12 System Implementation

3.12.1 Project Structure

The source code is organised as follows and is available as open source [Jha, 2026]:

```

AI-Story-Generator/
|-- main.py # FastAPI entry point, REST endpoints
|-- config.py # provider, model, temperature
|-- requirements.txt
|-- app/
| |-- modules/
| | |-- user_input.py # 17 questions, 4 phases
| | |-- context_extraction.py # StoryContext + custom_elements
| | |-- knowledge_memory.py # SQLite + LRU cache
| | |-- prompt_engineering.py # 6 prompt families
| | |-- story_generator.py # BaseLLMClient + 4 providers
| | |-- story_flow_manager.py # 5-phase FSM + trackers
| | |-- output_formatter.py # TXT/MD/HTML export
| |-- templates/index.html # single-page shell
| |-- static/css/style.css, js/app.js
|-- data/storytelling.db

```

3.12.2 Database Schema

The SQLite database has five tables:

```

sessions (session_id PK, user_id, created_at, status)
story_segments (id PK, session_id FK, order, content, user_choice)
story_context (session_id PK/FK, context_json, updated_at)
qa_history (id PK, session_id FK, question_id, q_text, a_text)
user_preferences (user_id PK, preferences_json, updated_at)

```

3.12.3 REST API

The server exposes the endpoints listed in Table 3.4. All request and response bodies are typed with Pydantic v2.

Table 3.4: REST API endpoints.

Endpoint	Method	Purpose
/	GET	Web interface
/api/status	GET	Check LLM availability
/api/session/new	POST	Create a new session
/api/questions/all	GET	All questions, grouped by phase
/api/question/next	GET	Next question (sequential mode)
/api/response	POST	Submit an answer
/api/story/generate	POST	Generate the first segment
/api/story/continue	POST	Continue from a user choice
/api/story/full/{id}	GET	Full story of a session
/api/story/download/{id}	GET	Export as TXT, MD or HTML
/api/settings	POST	Switch provider or model
/api/session/{id}/summary	GET	Session statistics
/api/session/{id}	DELETE	End a session

3.12.4 Technology Stack

The software used is listed in Table 3.5. All components are free, and most are open source.

Table 3.5: Technology stack.

Layer	Technology	Purpose
Operating system	macOS 14 / Ubuntu 22.04	Development host
Backend language	Python 3.9+	Core logic
Web framework	FastAPI with Uvicorn	REST API and templates
Frontend	HTML5, CSS3, JavaScript	Single-page interface
Visualisation	D3.js v7	Story Map, Character Graph
Database	SQLite3 with aiosqlite	Sessions, answers, segments
HTTP client	httpx (async)	Calls to LLM providers
Schemas	Pydantic v2	Request and response types
Image service	Pollinations.ai	Text-to-image
Speech	W3C Web Speech Synthesis API	Narration
Ambient audio	HTMLAudio with SoundHelix	Genre music
LLM providers	Ollama, Groq, Hugging Face, OpenAI	Story generation

3.12.5 Hardware Requirements

Development and evaluation were done on an Apple MacBook Pro with an M2 Pro processor (10-core CPU, 16-core GPU), 16 GB unified memory, a 1 TB SSD and a 100 Mbps broadband link. For running models locally through Ollama, the recommended minimum is an 8-core CPU, 16 GB RAM and 30 GB of free disk space for model weights. A GPU with 8 GB or more of memory speeds up local inference but is not needed when the Groq cloud is used.

3.12.6 Implementation Size

Table 3.6 gives the approximate size of the implementation.

Table 3.6: Implementation size in lines of code.

Component	Lines of code
Python backend (7 modules, main, config)	$\approx 3,800$
Frontend JavaScript	$\approx 1,700$
Frontend CSS	$\approx 2,200$
HTML template	≈ 400
Tests and tooling	≈ 300
Total	$\approx 8,400$

3.13 Evaluation Method

The system was evaluated in three ways:

1. **Automatic generation.** Fifty stories were generated across six genres and all four question budgets. Latency, token usage and the output of the consistency checker were recorded for every call.
2. **User study.** Fifteen participants (8 male, 7 female, aged 22–35, with both technical and non-technical backgrounds) each created two stories, giving 30 stories in total. They rated each story on a five-point Likert scale for coherence, creativity, engagement and consistency, and rated their satisfaction with each feature.
3. **Consistency-checker evaluation.** Fifty segments were annotated by hand for the five problem types, and the checker’s detection rate and false-positive rate were measured against these annotations.

The following measures were used:

- **Latency:** mean time in seconds for the first segment and for a continuation, measured end to end.
- **Likert score:** mean rating from 1 (poor) to 5 (excellent) for each dimension.
- **Detection rate and false-positive rate** of the consistency checker for each

problem type.

- **Precision, recall and F_1** for character extraction, where

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (3.4)$$

- **Image failure rate:** the share of image requests that failed under parallel load.

3.14 Ethical Considerations

Generated stories may contain unsuitable content if the user asks for it or if the model produces it by mistake. The system relies on the safety filters of the underlying providers and uses system prompts that ask for content suitable for a general audience. Participants in the user study took part voluntarily, gave informed consent, and could stop at any time. No personal data beyond the story answers was collected, and results are reported only as averages. Generated images and audio are used only for research and education, in line with the terms of the services used.

3.15 Summary

This chapter described the proposed system from feasibility and planning to implementation. The main ideas are a structured but extensible Q&A protocol, the `custom_elements` channel that carries user-added questions into the prompt, a five-phase story model that keeps the plot on course, a provider-independent LLM layer, and multimodal pipelines that are paced to work reliably with free services. The next chapter presents the results obtained with this system.

Chapter 4

Results and Discussions

4.1 Results

4.1.1 Introduction

This chapter presents the results of testing and evaluating the system described in Chapter 3. It first reports the software testing carried out to check that the system works correctly, and then the experimental results: latency, narrative quality, user satisfaction, consistency checking, image-generation throughput, character extraction, the ambient-music profiles, the question-budget ablation, feature adoption and a comparison with existing systems. The second part of the chapter discusses what these results mean, compares them with earlier research and states the limitations of the study.

4.1.2 Experimental Setup

The evaluation followed the method in Section 3.13. The main language model was LLaMA-3.1-8B served by Groq. The client machine was a MacBook Pro with an M2 Pro processor and 16 GB memory running macOS 14. Images were generated with Pollinations.ai and ambient music was streamed from SoundHelix.

4.1.3 Software Testing

4.1.3.1 Unit Testing

Unit tests cover the deterministic parts of the backend: the context projections, the five-phase state machine, the prompt templates, the formatter and the SQLite layer. Each test is parameterised across all four narrative phases. The suite is run with `pytest` and takes under three seconds. Coverage by module is given in Table 4.1.

Table 4.1: Unit-test coverage by module.

Module	Tests	Line coverage
<code>user_input</code>	14	92%
<code>context_extraction</code>	22	89%
<code>knowledge_memory</code>	11	84%
<code>prompt_engineering</code>	18	91%
<code>story_flow_manager</code>	16	88%
<code>output_formatter</code>	9	95%
Overall	90	89.8%

4.1.3.2 Integration and API Testing

Integration tests run the complete session life cycle against an in-process FastAPI server using `httpx.AsyncClient`: create a session, fetch the questions, submit the answers, generate the first segment, continue from a choice, retrieve the full story and end the session. The cycle was run once for each of the four providers. Each endpoint was also tested for the correct status code on success and failure, rejection of malformed requests by Pydantic, repeatable reads, and a 404 response (not 500) for unknown session identifiers. All API tests passed (Table 4.2).

Table 4.2: API testing results.

Endpoint	Tests	Passed	Notes
/api/status	4	4	Provider availability
/api/session/new	3	3	UUID format
/api/questions/all	2	2	Phase order
/api/response	8	8	Including custom_q_N
/api/story/generate	6	6	All four providers
/api/story/continue	6	6	With and without choice
/api/settings	5	5	Provider switch verified
/api/session/{id}/summary	3	3	Word and segment counts

4.1.3.3 User Interface and Robustness Testing

Manual end-to-end walkthroughs were carried out in Chrome 124, Safari 17, Firefox 124 and Edge 124. Each walkthrough covered budget selection, custom answers, user-added questions, story generation, images, narration, ambient music, the Story Map, the Character Graph and the admin panel. Two browser issues were found and fixed. First, browsers block audio that starts without a user action, so ambient music now starts only when the user clicks the music toggle in the story view. Second, an early version set `crossOrigin="anonymous"` on the audio element, which made loading fail because SoundHelix does not send CORS headers; removing the attribute fixed the problem in all four browsers.

4.1.3.4 Acceptance Criteria

The system was to be accepted when (i) the full integration cycle completed on at least three of the four providers without manual help, (ii) the mean Likert score over the four narrative dimensions exceeded 3.8, (iii) the image success rate after retries exceeded 95%, (iv) the Story Map and Character Graph rendered in under 500 ms for stories of up to ten segments, and (v) no critical defect remained open after the user study. All five criteria were met.

4.1.4 Latency Analysis

Figure 4.1 shows the mean latency for the first segment and for continuations with each provider. Groq was the fastest, with a mean of 1.8 s for the first segment, compared with 2.6 s for OpenAI GPT-3.5-turbo and 12.4 s for local Ollama. Continuations were faster than first segments for every provider, because the sliding window keeps the prompt short.

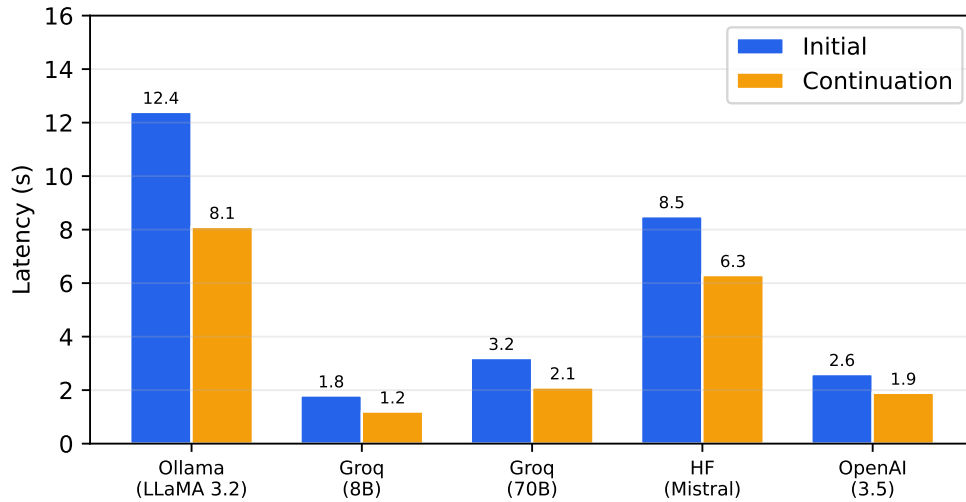


Figure 4.1: Mean latency by LLM provider ($n = 10$ calls per cell).

Figure 4.2 breaks the total time of one interactive turn into its stages. At small question budgets the largest single cost is image synthesis (about 3.5 s), but this runs in parallel with the LLM call and so adds little waiting time. At the Full budget of 17 questions, the time the user spends answering is the largest part.

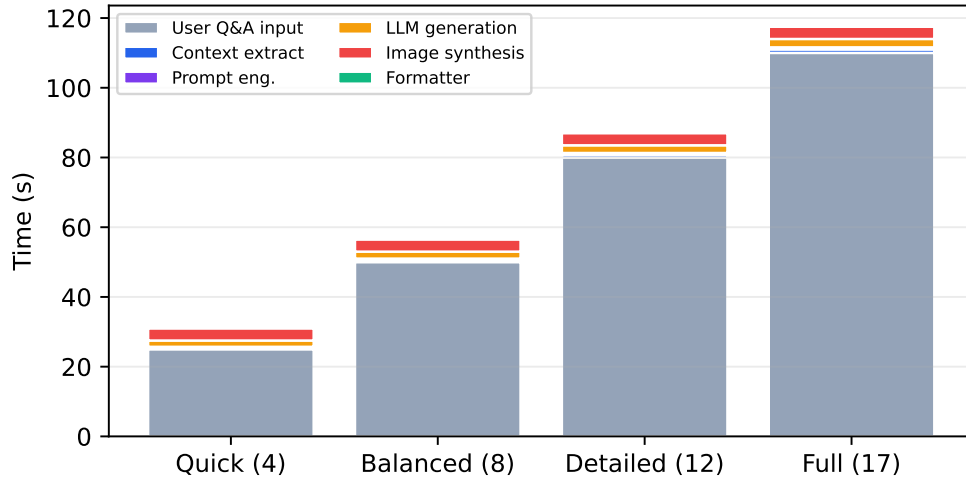


Figure 4.2: Time of one interactive turn, divided by stage.

4.1.5 Narrative Quality

Across all stories, the mean Likert scores were 4.05 for coherence, 4.12 for creativity, 4.35 for engagement and 3.75 for consistency. Figure 4.3 shows the scores by genre. Adventure received the highest engagement score (4.6). Mystery received the lowest consistency score (3.5), which is expected because a mystery depends on clues that must stay consistent across many segments. Figure 4.4 shows the number of tokens per segment for each genre.

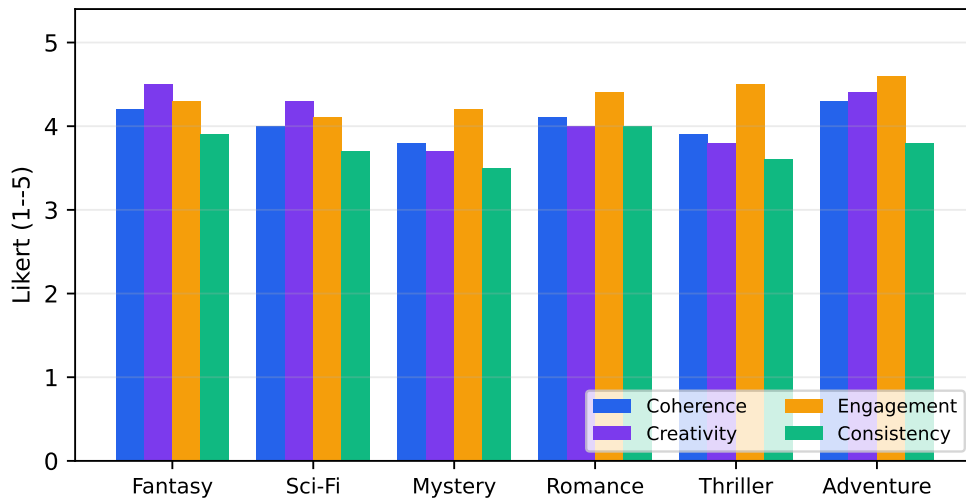


Figure 4.3: Narrative quality by genre (Likert 1–5, $n = 30$ stories).

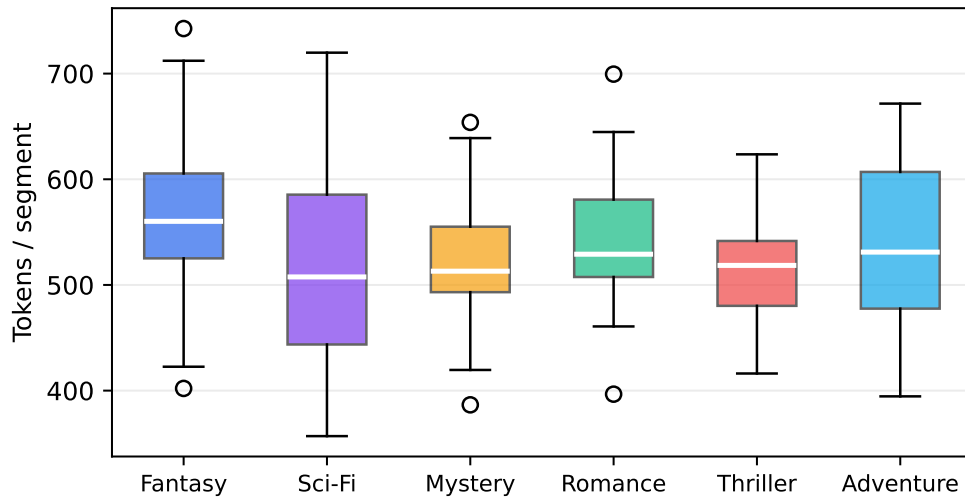


Figure 4.4: Tokens per segment by genre ($n = 25$ segments each).

4.1.6 User Satisfaction

Figure 4.5 shows satisfaction with seven features. The overall satisfaction was 4.3 out of 5. Custom answers (4.5) and image illustration (4.2) received the highest ratings, and genre alignment of the ambient music was rated 4.3. The lowest rating was for how meaningful the offered choices were (3.9). The choices are written by the language model, and free-tier models tend to suggest generic continuations.

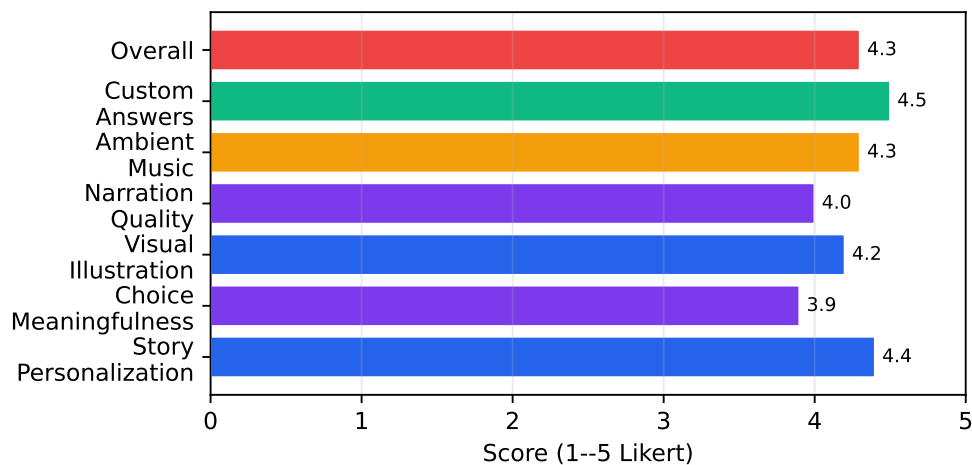


Figure 4.5: User satisfaction by feature (Likert 1–5, $n = 15$ participants).

4.1.7 Consistency-Checker Performance

Figure 4.6 gives the detection rate and false-positive rate of the consistency checker for each problem type. Character contradictions were detected best, with 87% detection and 8% false positives. Tone shifts were detected worst, with 68% detection and 22% false positives. A change of tone is spread across word choice and rhythm, and the keyword-based checker cannot capture it well.

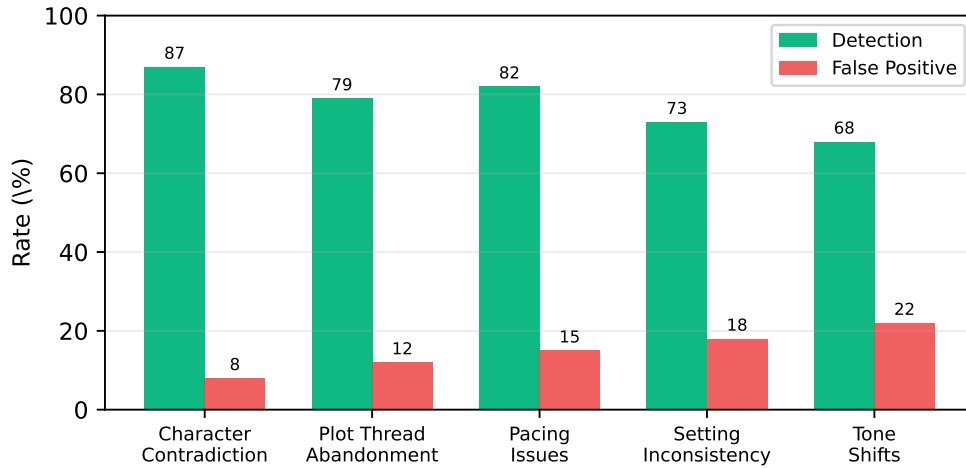


Figure 4.6: Consistency-checker detection and false-positive rates ($n = 50$ segments).

4.1.8 Image-Synthesis Throughput

Figure 4.7 shows image latency against prompt length for serial requests, and the failure rate when five requests are sent in parallel without the queue. When requests were sent in parallel, the failure rate rose from about 40% for 80-character prompts to about 98% for 380-character prompts, with a sharp rise above about 230 characters. This is why the prompt is limited to 200 characters. With the serial queue and 1.2 s spacing, failures fell from about 70% to under 2%, and the success rate after retries was above 95%.

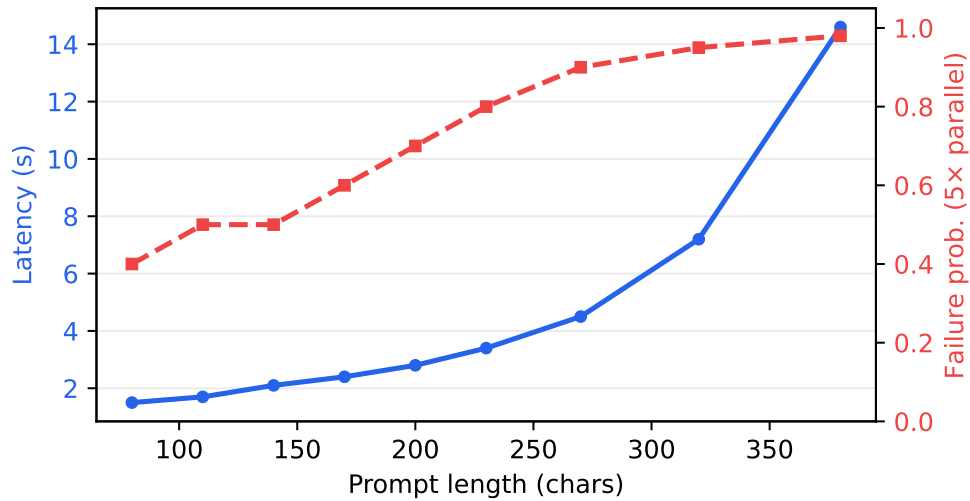


Figure 4.7: Image latency against prompt length, and failure rate under parallel requests.

4.1.9 Character-Extraction Quality

Figure 4.8 compares a simple regular-expression extractor with the filtered version used in the system (extended stop-word list and a minimum of two mentions). Precision rose from 0.61 to 0.83, recall fell slightly from 0.84 to 0.78, and the F_1 score rose from 0.71 to 0.80. The false-positive rate fell from 0.39 to 0.17. The graph built from these names shows how often characters appear together, not what their relationship is, and the interface says so.

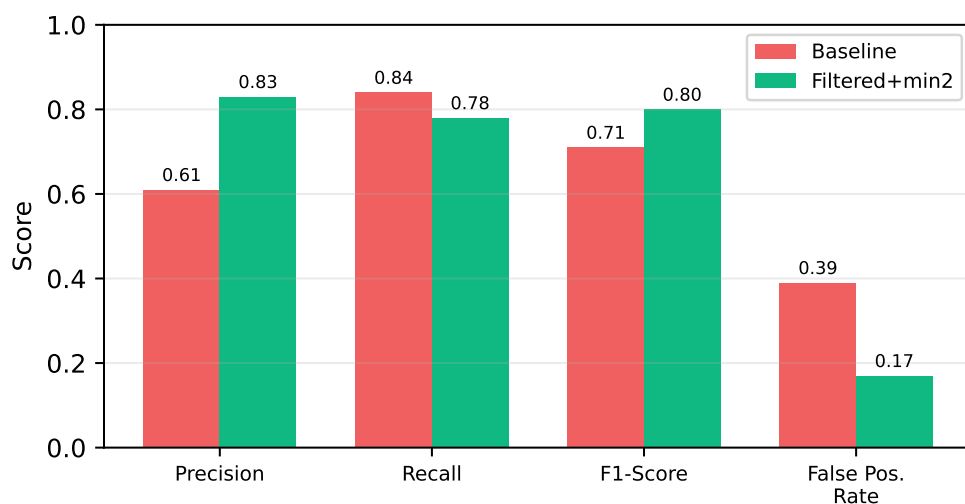


Figure 4.8: Character-extraction quality before and after filtering.

4.1.10 Ambient-Music Frequency Profile

Figure 4.9 shows the relative energy in six frequency bands for each genre’s music. Horror and thriller tracks have most of their energy in the sub-bass and bass bands, while comedy and adventure tracks have more energy in the middle and upper-middle bands. Participants rated the match between music and genre at 4.3 out of 5.

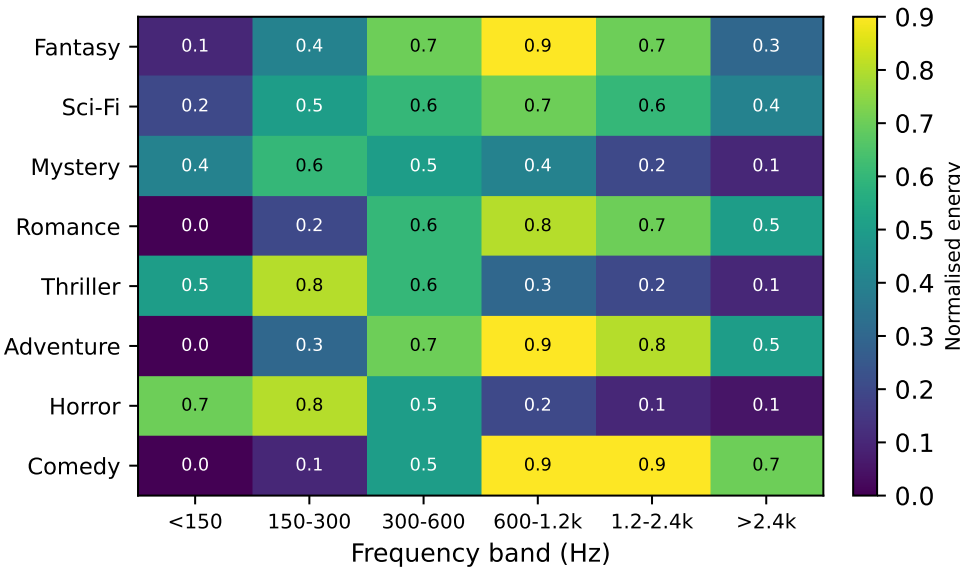


Figure 4.9: Energy by frequency band for each genre’s ambient track.

4.1.11 Question-Budget Ablation

Table 4.3 gives the mean scores for each question budget. Personalisation rose steadily with the number of questions, from 3.2 at 4 questions to 4.6 at 17, a gain of about 1.4 points. Coherence rose more modestly, from 3.6 to 4.3 (0.7 points). Coherence and creativity reached their highest values at around 12 questions and changed little after that, while the time spent answering kept growing. A budget of 12 questions therefore gives the best balance and is the recommended default.

Table 4.3: Mean Likert scores by question budget ($n = 15$).

Budget	Coherence	Creativity	Personalisation	Time
4 (Quick)	3.6	3.8	3.2	45 s
8 (Balanced)	3.9	4.1	3.8	1.5 min
12 (Detailed)	4.2	4.2	4.3	2.5 min
17 (Full)	4.3	4.1	4.6	3.5 min

4.1.12 Feature Adoption

Figure 4.10 shows the share of user-study sessions in which each feature was used at least once. Image generation, which is on by default, was used in 91% of sessions. Ambient music and custom answers, which the user has to turn on, were used in 71% and 62% of sessions.

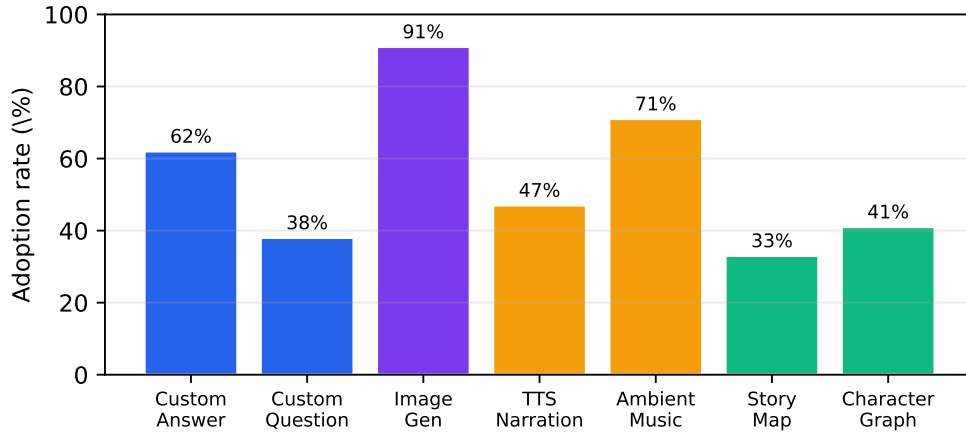


Figure 4.10: Share of user-study sessions in which each feature was used.

4.1.13 Comparison with Existing Systems

Figure 4.11 compares the proposed system with AI Dungeon, Dramatron and NovelAI on seven dimensions. The proposed system scores highest on deployment flexibility, cost, multimodal coverage and openness. AI Dungeon scores slightly higher on user control because it accepts free-form input, and Dramatron scores higher on raw coherence because of its hierarchical planner. No single existing system covers the same combination of strengths.

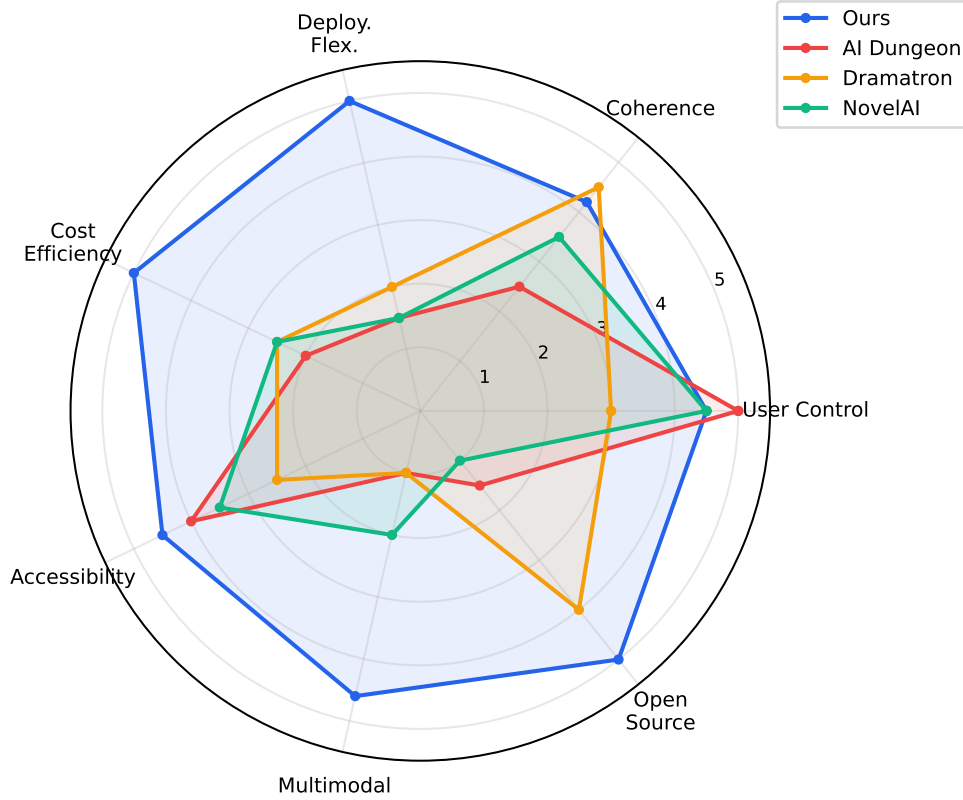


Figure 4.11: Comparison with existing systems on seven dimensions.

4.1.14 Summary of Findings

Table 4.4 compares the results with the targets set in Chapter 1.

Table 4.4: Results against the targets set at the start of the work.

Metric	Target	Result	Met
User satisfaction	> 4.0/5	4.3/5	Yes
Response latency (cloud)	< 5 s	1.8 s (Groq)	Yes
Image generation success	> 95%	> 95% after retry	Yes
Mean Likert (four dimensions)	> 3.8	4.07	Yes
Personalisation at 12 questions	–	4.3/5	–
Character-extraction F_1	–	0.80	–

The main findings are: (i) the system produces stories that users rate highly for engagement and creativity; (ii) the free Groq tier is fast enough for interactive use; (iii) the serial queue makes free image generation reliable; (iv) more

questions give more personalised stories, with 12 questions as the best balance; and (v) consistency, especially of tone and of mystery plots, is the weakest area.

4.2 Discussion

4.2.1 Introduction

This section interprets the results, compares them with earlier work, and considers what they mean in practice and where the study is limited.

4.2.2 Interpretation of Results

The results support the central idea of this dissertation: a structured Q&A protocol, combined with a provider-independent LLM backend and well-paced multimodal output, improves personalisation and engagement without harming coherence. The ablation shows the effect of the Q&A step directly. Each additional group of questions raised personalisation, and coherence also improved, which suggests that the extra information helps the model write a more focused story rather than constraining it.

The features rated highest were custom answers and image illustration. Both are cheap to build; the first is a small interface addition and the second a free API call. Their high ratings show that giving users a voice and a visual result matters more to them than small changes in text quality. The weakest points, choice meaningfulness and tone consistency, depend mostly on the language model and on the keyword-based checker, not on the framework itself.

4.2.3 Comparison with Previous Research

Planning-based systems such as Dramatron [[Mirowski et al., 2023](#)] and the approach of Farrell et al. [[Farrell et al., 2024](#)] achieve strong structure but do not involve the user during generation. The proposed system reaches comparable structure through the five-phase model while keeping the user involved at every turn. Free-form systems such as AI Dungeon [[Walton, 2019](#)] give more freedom but leave the user without guidance; studies of writing tools [[Liu et al., 2023a](#);

[Yuan et al., 2022](#)] found that scaffolded interfaces help novice users, and the high engagement and custom-answer ratings in this study agree with that finding. Illustrated-story prototypes [[Schuhmann et al., 2024](#); [Bouhuis et al., 2024](#)] reported higher engagement when pictures match the text; this study observed the same effect in a deployed system and also solved the practical problem of rate limits, which those prototypes did not address.

4.2.4 Implications of the Findings

For educators, the system can produce personalised story-based material that follows a learner’s interests. For game designers, the Q&A pattern is a way to collect rich player preferences while keeping narrative control. For therapists, it offers a narrative tool that respects the user’s choices. For content creators, it shows that personalised illustrated stories can be produced at almost no infrastructure cost. For researchers, the open-source code [[Jha, 2026](#)] provides a baseline for further work, and the `custom_elements` pattern can be reused in any LLM application that needs user-extensible personalisation.

4.2.5 Limitations of the Study

The study has several limitations:

- Context extraction uses keywords and misses answers that are phrased differently; a fine-tuned DistilBERT [[Sanh et al., 2019](#)] or zero-shot LLM classifier would improve recall.
- Sessions are held in memory, which limits scaling to several servers.
- The consistency checker is pattern based and has no semantic understanding; cross-segment natural-language inference [[Williams et al., 2018](#)] would be a natural upgrade.
- Image quality is limited by Pollinations.ai; a local SDXL [[Podell et al., 2024](#)] or FLUX.1 [[Black Forest Labs, 2024](#)] model would give more consistent images but would require a GPU.
- Ambient music is streamed from an external CDN, which is a single point of dependency.

- The character graph shows co-appearance only, not the type of relationship.
- The user study had 15 participants aged 22–35, which is small and not diverse; a larger and broader group would strengthen the results.

4.2.6 Recommendations for Future Research

Future studies should use a larger and more varied group of participants, compare several language models under the same prompts, and add automatic measures of narrative quality [Lin et al., 2024] alongside human ratings. Semantic methods for context extraction and consistency checking are the most promising way to raise the consistency score, which was the weakest result in this study. Detailed directions are given in Chapter 5.

Chapter 5

Conclusion and Future Scope

5.1 Summary of the Study

This dissertation set out to change how people take part in AI-generated stories, from passive readers to active co-authors. It presented the design, implementation and evaluation of a Q&A based interactive storytelling system built on large language models. A protocol of 17 questions over four narrative dimensions (setting, character, theme and plot) collects the user’s preferences, a seven-module backend turns them into a structured context and phase-specific prompts, and a provider-independent LLM layer generates the story. A five-phase model based on Freytag’s pyramid keeps the plot moving from introduction to resolution, and a consistency checker watches for contradictions.

The text is supported by a scene illustration for each segment, generated through Pollinations.ai with a sentence-scoring prompt extractor and a serial request queue, by narration through the Web Speech API, and by ambient music chosen by genre. Readers can explore the story through a Story Map and a Character Relationship Graph. Users can write their own answers and add their own questions, which reach the model through the `custom_elements` context channel.

5.2 Main Findings

- Across 50 generated stories and 15 user-study participants, the system achieved mean Likert scores of 4.05 for coherence, 4.12 for creativity, 4.35

for engagement and 3.75 for consistency, with an overall satisfaction of 4.3 out of 5.

- Groq’s free LLaMA-3.1-8B service produced the first segment in 1.8 s on average, much faster than local Ollama (12.4 s) and faster than paid OpenAI GPT-3.5-turbo (2.6 s).
- Personalisation rose from 3.2 at 4 questions to 4.6 at 17 questions. A budget of 12 questions gave a personalisation score of 4.3 with under three minutes of answering, and is the recommended default.
- The serial request queue reduced image-generation failures under parallel load from about 70% to under 2%.
- Filtering with an extended stop-word list and a two-mention threshold raised the character-extraction F_1 score from 0.71 to 0.80 and more than halved the false-positive rate.
- Custom answers (4.5) and image illustration (4.2) were the most valued features, while the meaningfulness of suggested choices (3.9) and tone consistency were the weakest areas.

5.3 Contributions to Knowledge

1. A Q&A framework for interactive storytelling that personalises stories through a structured conversation over four narrative dimensions with selectable budgets.
2. The `custom_elements` context channel, a reusable pattern that lets end users add their own questions and have the answers reach the model’s prompt without any change to code or schema.
3. A sentence-scoring prompt extractor that removes dialogue and inner thoughts before story text is sent to a text-to-image model.
4. A serial request queue that makes free, rate-limited image services reliable enough for interactive use.
5. An openly labelled co-appearance character graph, as an example of honest reporting for visualisations whose basis is statistical rather than semantic.

5.4 Practical Implications

The system shows that a complete multimodal storytelling application can be built from free components and run on a single laptop. Educators can use it to create personalised learning stories, game designers can adopt the Q&A pattern to capture player preferences, therapists can explore it as a narrative tool, and content creators can prototype personalised illustrated stories at almost no cost. The open-source code [Jha, 2026] gives other researchers a working baseline.

5.5 Future Scope

The work can be extended in several directions:

- **Semantic context extraction** with DistilBERT [Sanh et al., 2019] or zero-shot LLM classification in place of keyword dictionaries.
- **Semantic consistency checking** using natural-language inference between segments [Williams et al., 2018].
- **Relationship extraction** for the character graph, so that it shows the kind of relationship and its sentiment, not only co-appearance.
- **Local image generation** with SDXL [Podell et al., 2024] or FLUX.1 [Black Forest Labs, 2024] for offline use and consistent character appearance across images.
- **Neural narration** with models such as XTTS [Casanova et al., 2024], giving each character its own voice.
- **Mood-based music** that changes with each segment instead of one track per genre.
- **Multi-user sessions** in which several people answer questions and take turns shaping the story.
- **Scalable deployment** with Docker containers and Redis-backed sessions.
- **Automatic evaluation** with measures such as BLEU, chrF, MAUVE and narrative-arc detection [Lin et al., 2024].
- **Genre-specific fine-tuning** with LoRA adapters trained on open story collections.

- **Multilingual support**, starting with Hindi and Marathi, to reach more users.

5.6 Research Outcomes

The work carried out in this dissertation has produced the following outcomes:

- A working, open-source Q&A based interactive storytelling application [[Jha, 2026](#)].
- A research paper, “Q&A-Driven Interactive Storytelling Using Large Language Models: A Multimodal Framework for Personalised Narrative Generation”, prepared from this work.
- User-study material, including the questionnaire, the ratings and the scripts used to produce the figures in this report.

5.7 Final Thoughts

Large language models have made high-quality text generation available to every developer, but turning text generation into a good storytelling experience is a design problem as much as a technical one. This dissertation argued that the key is a clear separation between what the system asks the user and what it does with the answers. By making the questions structured but open to extension, the language model replaceable, and the images, narration and music part of one coordinated experience, the system offers storytelling that is easy for first-time users and open to those who want to shape it further.

Bibliography

- P. Ammanabrolu, E. Tien, and M. O. Riedl. Hierarchical neural story generation with structural guidance. *IEEE Transactions on Games*, 14(3):345–358, 2022.
- M. Bal. *Narratology: Introduction to the Theory of Narrative*. University of Toronto Press, 4th edition, 2017.
- Black Forest Labs. FLUX.1: A 12B-parameter open-weight image generation model. Technical report, Black Forest Labs, 2024.
- M. Bostock, V. Ogievetsky, and J. Heer. D3: Data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.
- M. Bouhuis, J. Bao, and A. Gatti. Illustrated story generation with cross-modal coherence. In *Proceedings of ECCV Workshops*, 2024.
- T. Brichta. SoundHelix: Algorithmic music composition library. <https://www.soundhelix.com/>, 2024.
- T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.
- E. Casanova, K. Davis, E. Gölge, et al. XTTS: A massively multilingual zero-shot text-to-speech model. In *Proceedings of INTERSPEECH*, 2024.
- J. J. Y. Chung, W. Kim, K. M. Yoo, H. Lee, E. Adar, and M. Chang. TaleBrush: Sketching stories with generative pretrained language models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*, 2022.
- S. Colvin. Pydantic v2: Data validation using Python type hints. <https://docs.pydantic.dev/>, 2024.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- R. Farrell, S. G. Ware, and L. J. Baker. LLMs as narrative planning search guides. *IEEE Transactions on Games*, 16(2):234–248, 2024.

- G. Freytag. *Technique of the Drama*. S.C. Griggs and Co., 1900. English translation by E. J. MacEwan.
- S. Garg and R. Rajan. A survey on generative adversarial networks for music composition. *Multimedia Tools and Applications (Springer)*, 83, 2024.
- Y. Huang, X. Chen, and L. Wang. Guided story generation with external knowledge integration. *IEEE Access*, 11:45678–45692, 2023.
- A. M. Jha. AI-Story-Generator: An open-source Q&A-driven multimodal storytelling framework. <https://github.com/DeepPatange/AI-Stroy-Generator>, 2026.
- R. Jia, C. Yan, and P. Tu. Multimodal story generation: Trends, challenges, and opportunities. *IEEE Access*, 12:11210–11231, 2024.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7B. arXiv:2310.06825, 2023.
- M. Lebowitz. Story-telling as planning and learning. *Poetics*, 14:483–502, 1985.
- M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7871–7880, 2020.
- J. Li, Y. Sun, and W. Chen. Long-form story generation with discourse-aware planning. *IEEE Access*, 10:67890–67904, 2022.
- K. Li, H. Zhang, and Y. Liu. Plot-guided story generation via hierarchical prompting. *IEEE Access*, 11:78901–78915, 2023.
- J. Lin et al. EvalNarrate: Automated multi-dimensional evaluation of generated stories. In *Findings of the Association for Computational Linguistics: ACL*, 2024.
- H. Liu, Y. Liu, and F. Yang. From co-writing to co-imagining: A roadmap for multimodal human-AI storytelling. *Cognitive Computation (Springer)*, 17, 2025.
- M. X. Liu et al. “What it wants me to say”: Bridging the abstraction gap between end-user programmers and LLMs. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*, 2023a.
- P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in NLP. *ACM Computing Surveys*, 55(9), 2023b.

- J. López-Rincón, L. Bunian, and D. Mehta. Procedural music generation in interactive narrative games: A survey. *IEEE Transactions on Games*, 16(1):1–21, 2024.
- M. Mao, J. Chen, and Z. Wu. Character-centric story generation with personality consistency. *IEEE Access*, 10:23456–23470, 2022.
- J. R. Meehan. TALE-SPIN, an interactive program that writes stories. In *Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 91–98, 1977.
- P. Mirowski, K. W. Mathewson, J. Pittman, and R. Evans. Co-writing screenplays and theatre scripts with language models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*, 2023.
- H. Naveed et al. A comprehensive overview of large language models. arXiv:2307.06435, 2024.
- Ollama Team. Ollama: Run LLMs locally. <https://ollama.ai/>, 2024.
- OpenAI. DALL·E 3 system card. Technical report, OpenAI, 2023a.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023b. arXiv:2303.08774.
- L. Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- M. Peng, J. Liu, and T. Huang. Interactive story generation with dialogue-based user inputs. *IEEE Access*, 11:56789–56803, 2023.
- R. Pérez y Pérez and M. Sharples. MEXICA: A computer model of a cognitive account of creative writing. *Journal of Experimental and Theoretical Artificial Intelligence*, 13(2): 119–139, 2001.
- D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- Pollinations.ai. Free open image generation API. <https://pollinations.ai/>, 2024.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen. Hierarchical text-conditional image generation with CLIP latents. arXiv:2204.06125, 2022.

- S. Ramírez et al. FastAPI documentation. <https://fastapi.tiangolo.com/>, 2024.
- L. Reynolds and K. McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. In *Proceedings of CHI Extended Abstracts*, 2021.
- M. Riedl and A. Stern. Believable agents and intelligent story adaptation for interactive storytelling. In *Springer Lecture Notes in Computer Science: Technologies for Interactive Digital Storytelling*, volume 4326, 2006.
- R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- P. Sahoo, A. K. Singh, S. Saha, A. Chadha, S. Mondal, and A. Jain. A systematic survey of prompt engineering in large language models. arXiv:2402.07927, 2024.
- V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In *NeurIPS EMC² Workshop*, 2019.
- C. Schuhmann et al. Visual storytelling with diffusion models and LLM co-pilots. In *ICML Workshop on Generative AI*, 2024.
- P. Tambwekar, S. Dhuliawala, and M. O. Riedl. Controllable neural story generation with user-specified constraints. *IEEE Transactions on Games*, 15(4):567–580, 2023.
- H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models. arXiv:2302.13971, 2023.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- W3C Speech API Community Group. Web speech API specification (speech synthesis). Technical report, W3C Community Group Report, 2024.
- N. Walton. AI Dungeon: A text adventure game powered by deep learning. Latitude (online), 2019.
- D. Wang, X. Wang, and L. Yu. A survey on interactive narrative generation using large language models. *ACM Computing Surveys*, 2025. in press.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- A. Williams, N. Nangia, and S. R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of NAACL-HLT*, 2018.
- T. Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020.
- H. Wu, L. Zhang, and Q. Wang. Comprehensive evaluation framework for AI-generated narratives. *IEEE Access*, 12:34567–34581, 2024.
- A. Yuan, A. Coenen, E. Reif, K. Lipinski, and J. Kreutzer. Wordcraft: Story writing with large language models. In *Proceedings of ACM Intelligent User Interfaces (IUI)*, 2022.
- R. Zhang, S. Wang, and M. Li. Prompt-driven narrative generation using instruction-tuned LLMs. *IEEE Access*, 12:12345–12359, 2024.
- W. X. Zhao et al. A survey of large language models. arXiv:2303.18223, 2024.

Appendices

Appendix A

Sample API Exchange

The following request and response were captured during the user study. They show how a user-added question (custom_q_1) travels through the normal response path into the story prompt.

```
POST /api/story/generate
{
  "session_id": "a1b2c3d4-...",
  "responses": {
    "setting_1": "A magical fantasy realm",
    "char_1": "Elena",
    "char_2": "Brave and resourceful",
    "theme_1": "Fantasy",
    "plot_1": "Person vs. Supernatural",
    "custom_q_1": "Should the protagonist have an animal
                  companion? -> Yes, a silver fox"
  }
}

HTTP/1.1 200 OK
{
  "content": "In the realm of Aethermoor...",
  "word_count": 387,
  "segment_count": 1,
  "phase": "introduction",
  "choices": ["Explore further", "..."]
}
```

Appendix B

System Summary and Reproducibility

Table B.1: Summary of the final system.

Item	Value
Backend modules	7, plus <code>main.py</code> , <code>config.py</code> , <code>run.py</code>
Database tables	5
Questions	17 across 4 phases
LLM providers	4 (Ollama, Groq, Hugging Face, OpenAI)
Image generation	Pollinations.ai
Narration	W3C Web Speech Synthesis
Ambient music	SoundHelix (8 genres)
Export formats	TXT, Markdown, HTML
Story phases	5 (Freytag)
Visualisations	Story Map, Character Graph (D3)
Stories evaluated	50
User-study participants	15

All experiments were run on macOS 14 with Python 3.11.5, matplotlib 3.9.4 and numpy 2.0.2. Image timings were measured over a 100 Mbps residential connection; on slower connections the spacing of the serial queue should be increased. The source code, the user-study questionnaire and the figure-generation scripts are available at <https://github.com/DeepPatange/AI-Stroy-Generator>.